Actividad N°3 – SimuPi – Node-RED – MQTT en AWS

Grupo 03

Integrantes:

- Daltroy Mateo
- Lima Francisco
- Lamiral Matheo

Contexto inicial

El objetivo de la actividad es reemplazar el bróker de AWS IoT de la Actividad 2 por el bróker MQTT embebido de **SimuPi**, un simulador de Raspberry Pi que genera sensores, pines GPIO y mensajes MQTT. Trabajamos con dos instancias EC2 **t3.micro** con **Debian 13**: en la **Instancia A** corre SimuPi y en la **Instancia B** corre Node-RED con un dashboard que lee y controla la simulación por MQTT.

Las Partes 1 y 2 se hicieron en AWS durante la clase. Al quedarnos sin cupo de vCPU en Ohio (us-east-2), la Instancia A terminó en la región São Paulo (sa-east-1), lo que nos obligó a adaptar algunos puntos de la consigna; lo explicamos en cada paso. La Parte 3 se desarrolló después, con las instancias ya terminadas, reemplazando cada instancia por un contenedor Docker en una computadora del grupo.

Parte 1 – Infraestructura

1) Instancia B – Node-RED (Ohio)

Desde la consola de EC2 lanzamos la instancia `G03-NodeRED` con AMI **Debian 13**, tipo **t3.micro** y el grupo de seguridad del grupo que ya teníamos, `Grupo_03_security_group`. En *Advanced details → User data* pegamos el script de la cátedra `userdata_instancia_B_nodered.sh`, que en el primer arranque instala Docker y Docker Compose, levanta Node-RED en un contenedor publicado en el puerto 1880 e instala el paquete `@flowfuse/node-red-dashboard`.

El primer intento falló porque la cuenta llegó al límite de **16 vCPU** en la región: todavía teníamos corriendo una instancia anterior (la habíamos creado sin el script, para no tener que pasar el archivo por SSH). La terminamos y el lanzamiento funcionó.

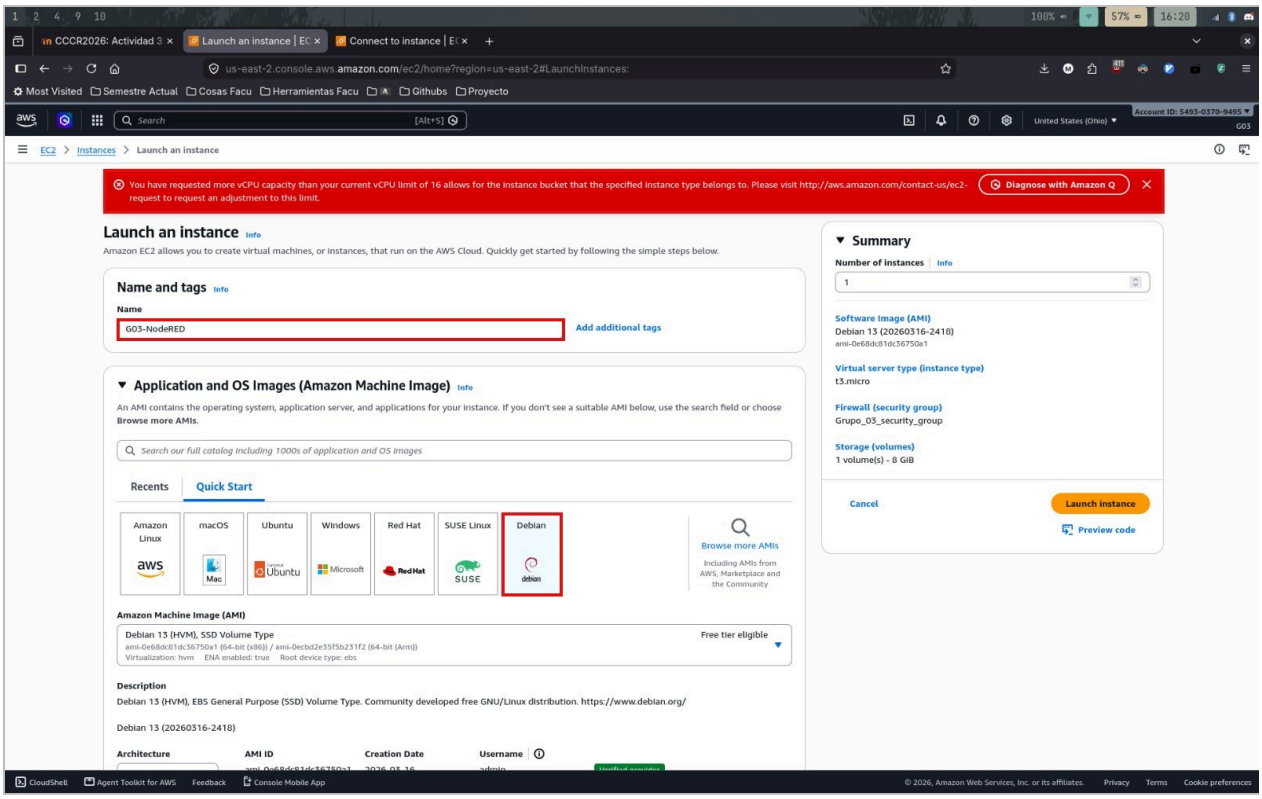


Figura 1: lanzamiento de G03-NodeRED (Debian 13, t3.micro) y error por el límite de vCPU de la cuenta.

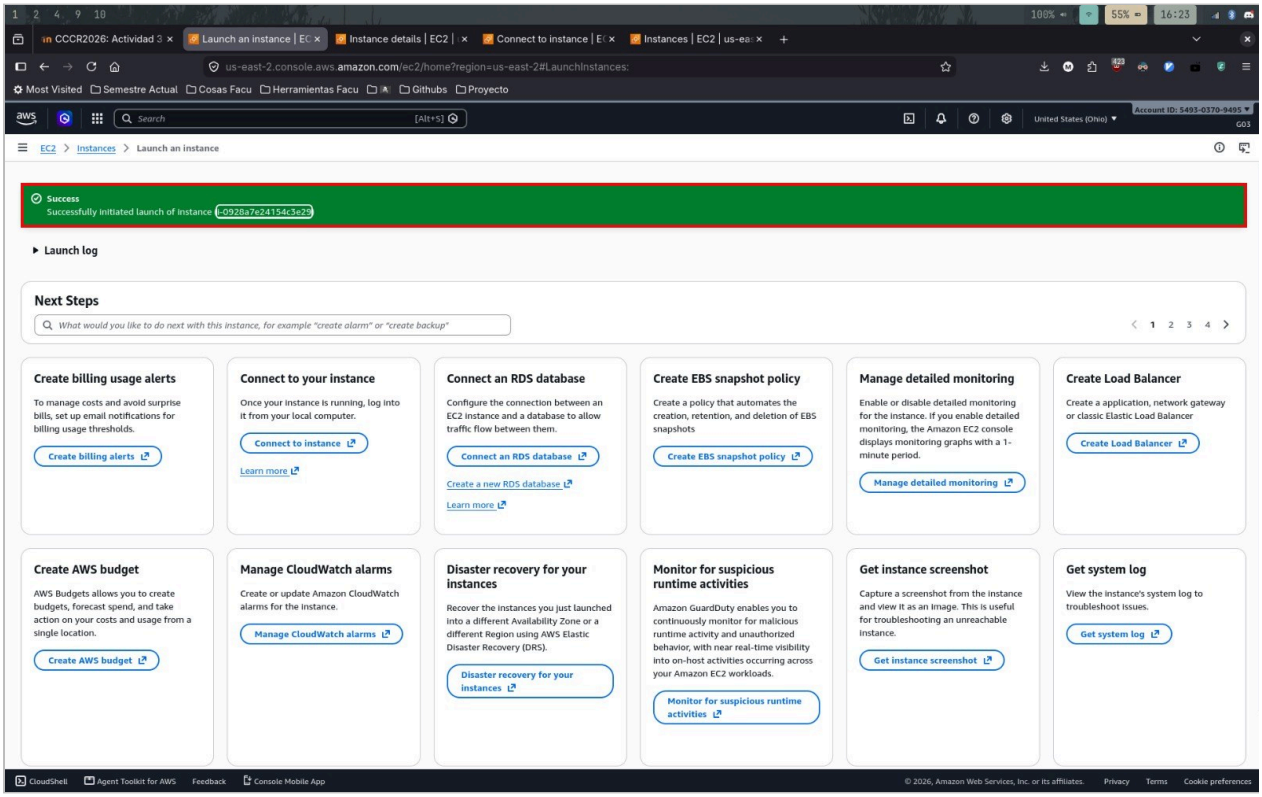


Figura 2: lanzamiento exitoso de la Instancia B después de terminar la instancia anterior.

Nos conectamos por SSH e instalamos `net-tools`. Con `sudo ss -tulpn` comprobamos que el contenedor de Node-RED quedó escuchando en el puerto **1880** (proceso `docker-proxy`), y desde el navegador entramos al editor con la IP pública de B, `3.129.25.11:1880`.

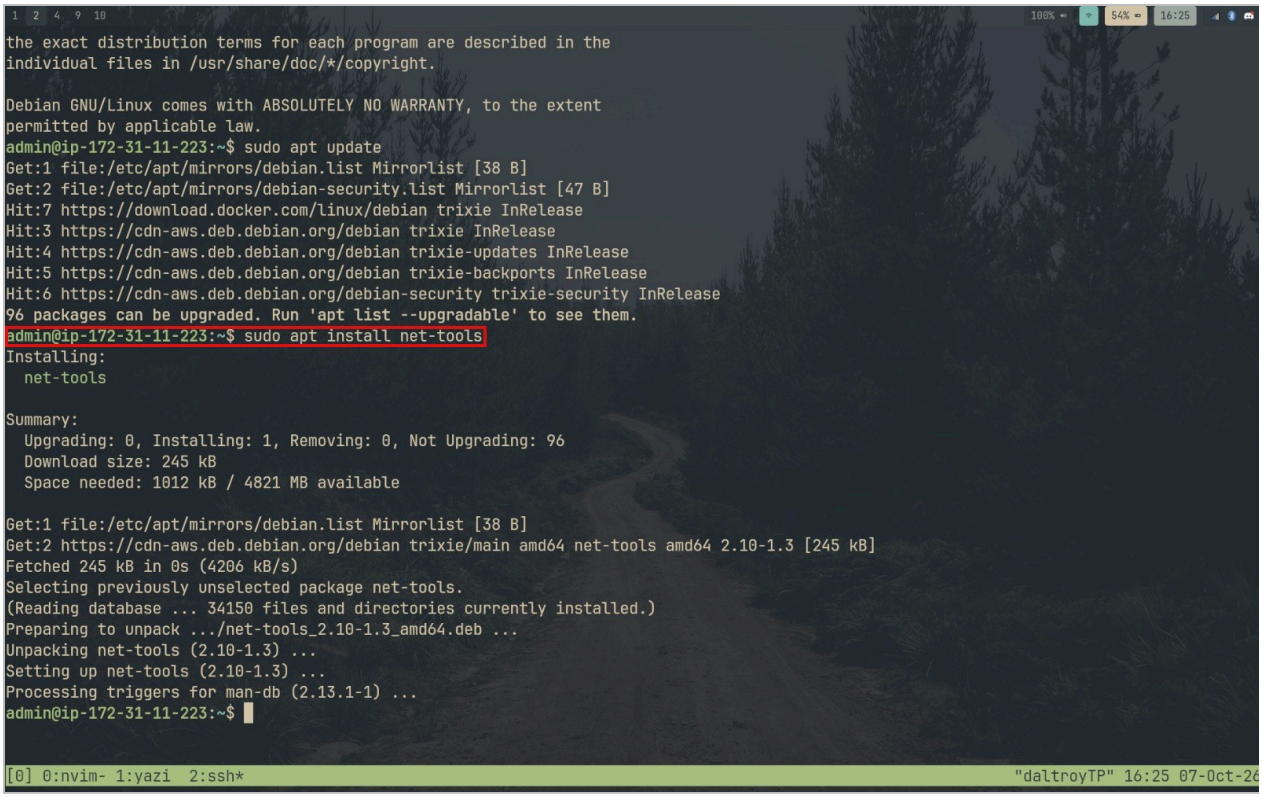


Figura 3: instalación de net-tools en la Instancia B.

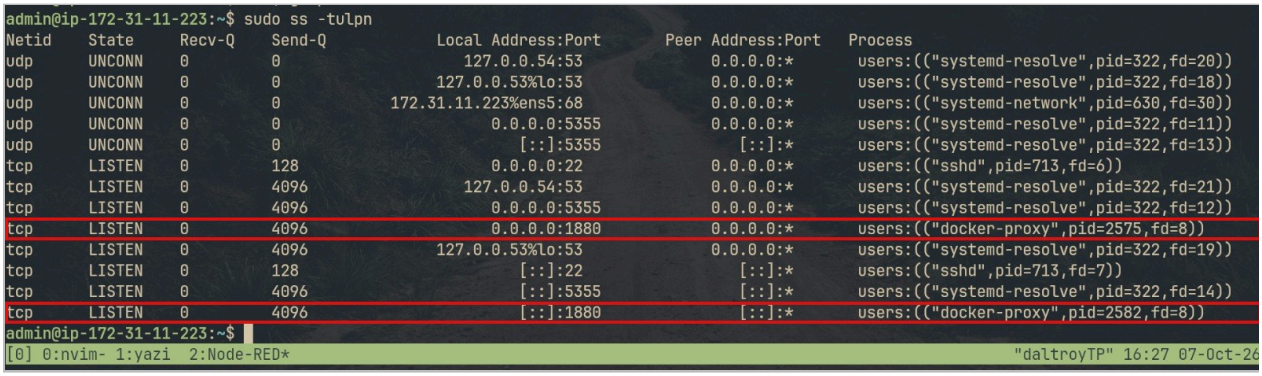


Figura 4: Node-RED escuchando en el puerto 1880 a través de docker-proxy.

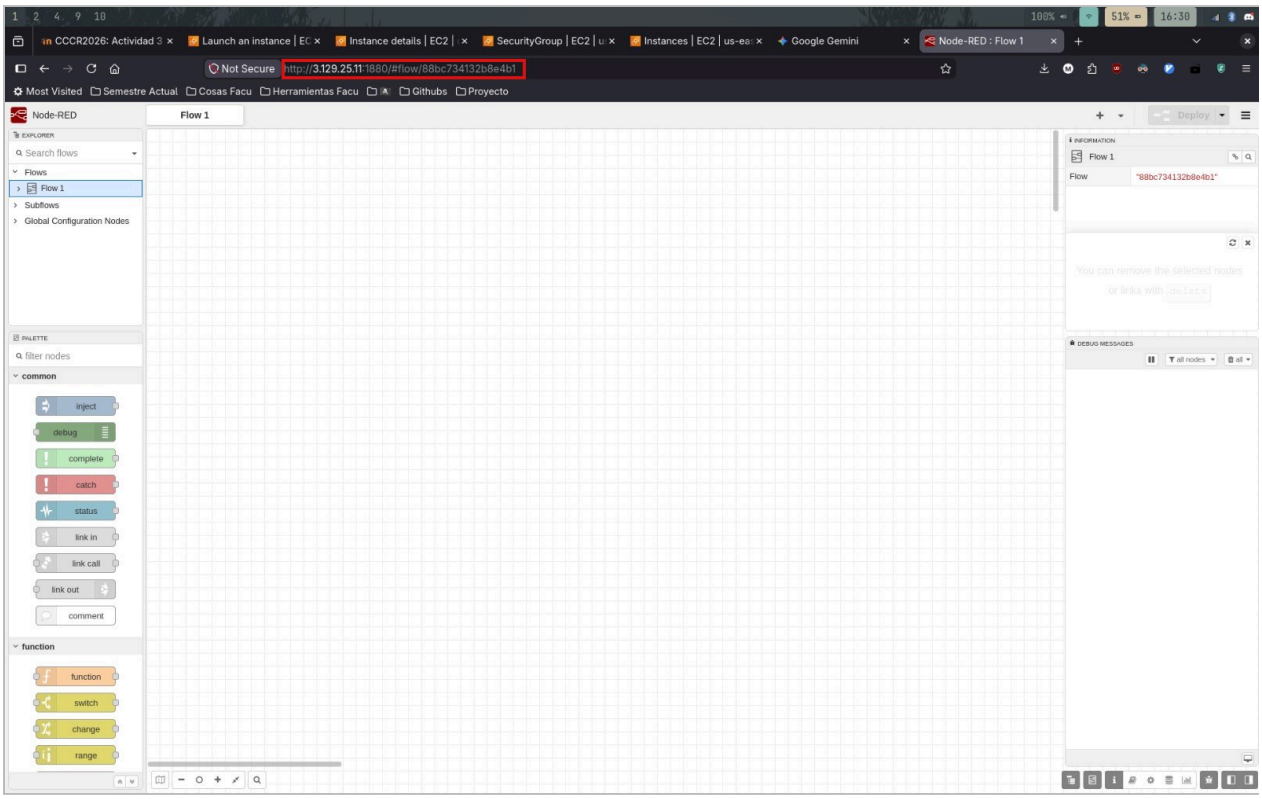


Figura 5: editor de Node-RED de la Instancia B, accedido por su IP pública.

2) Grupo de seguridad de la Instancia B

El grupo `Grupo_03_security_group` venía de prácticas anteriores con SSH (22) y Node-RED (1880) abiertos a `0.0.0.0/0`. La consigna pide que los dos puertos se permitan solo desde la IP del alumno. Para saber qué IP cargar ejecutamos `ip a` en nuestra computadora: la dirección es `163.10.143.79` dentro de la red `/22` de la facultad.

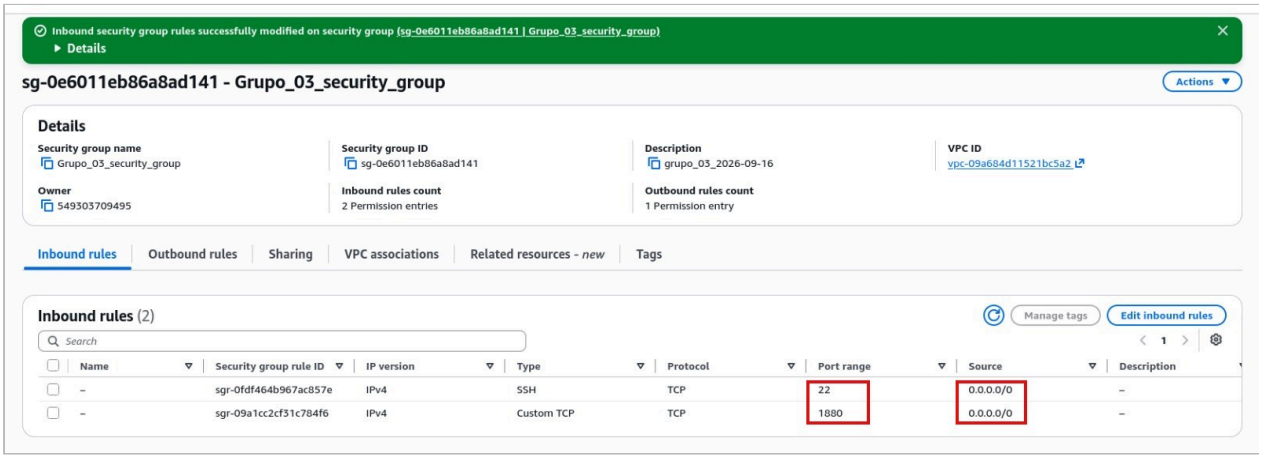


Figura 6: reglas iniciales de la Instancia B: puertos 22 y 1880 abiertos a 0.0.0.0/0.

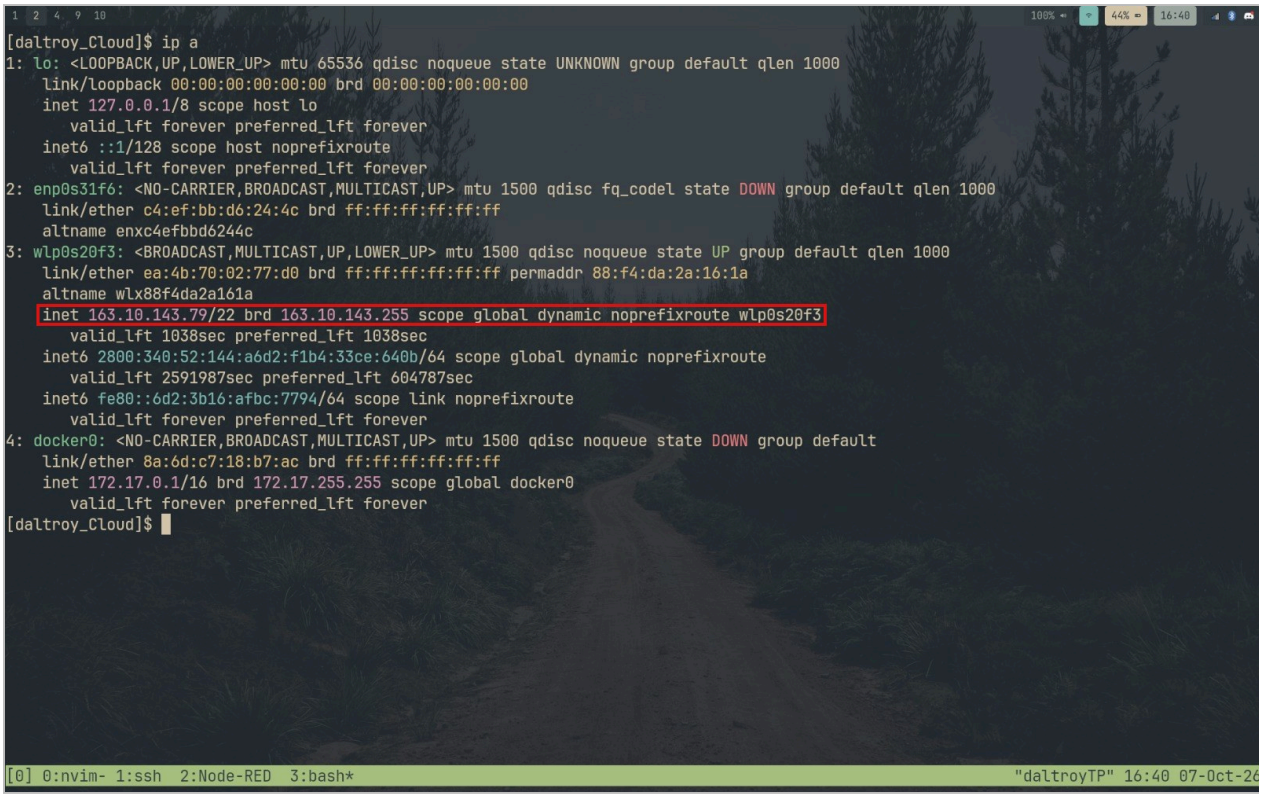


Figura 7: IP de nuestra computadora en la red de la facultad (163.10.143.79/22).

Editamos las dos reglas para restringir el origen. Primero cargamos el bloque `163.10.140.0/22` de la red de la facultad, desde la que trabajamos todos los integrantes, y después lo limitamos a nuestra IP exacta (`163.10.143.79/32`), como pide la consigna.

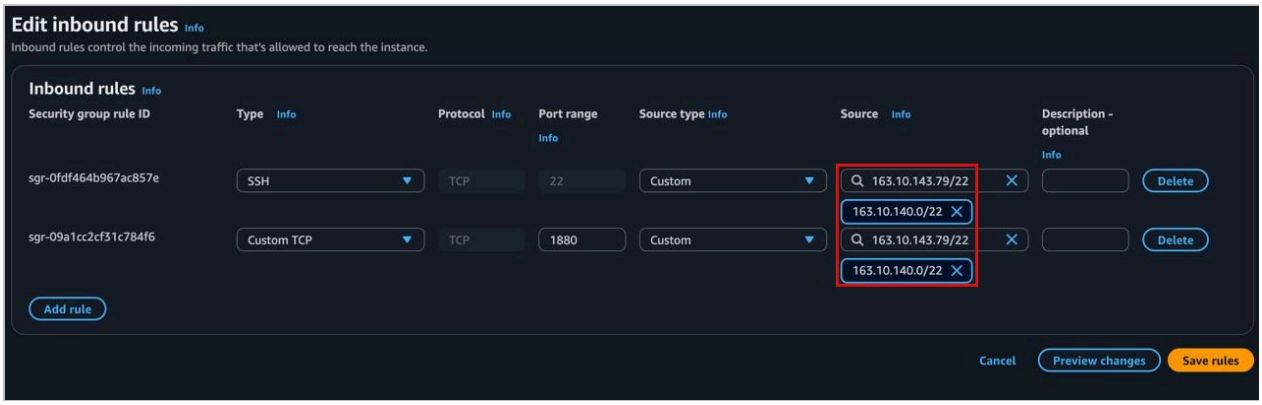


Figura 8: reglas de B con el bloque /22 de la red de la facultad.

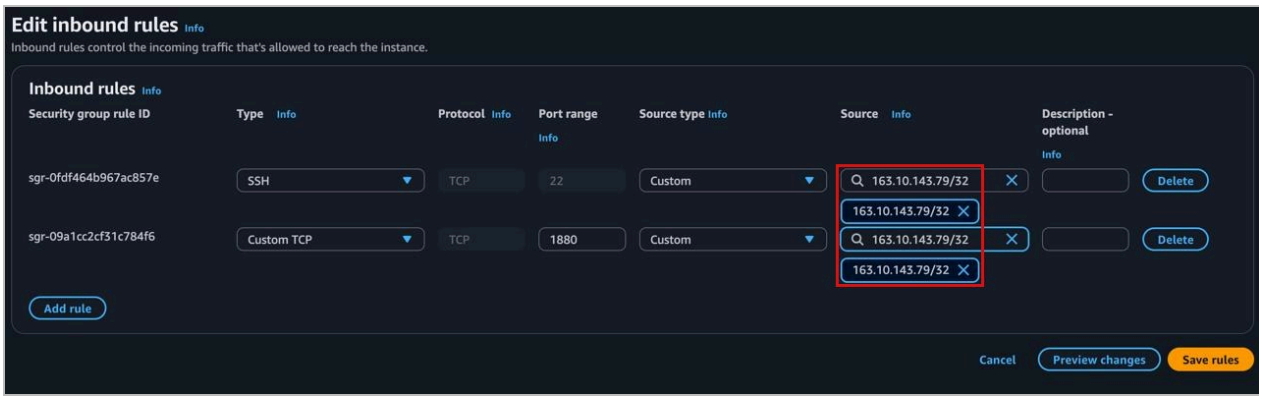


Figura 9: reglas de B limitadas a nuestra IP (/32), su configuración final.

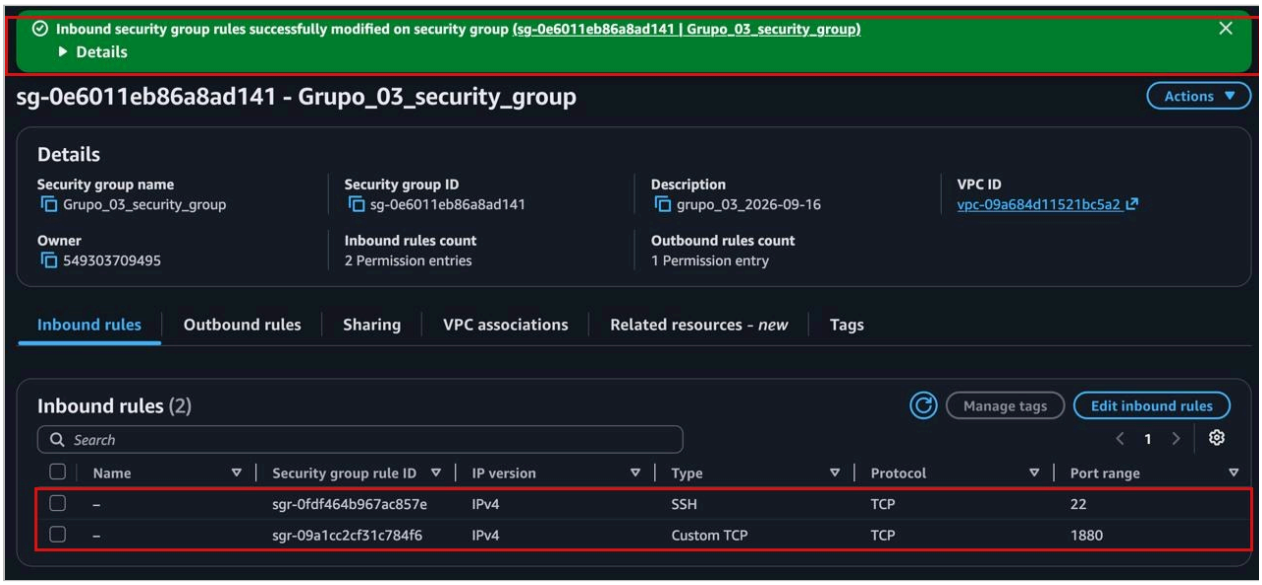


Figura 10: reglas de entrada de la Instancia B guardadas.

3) Instancia A – SimuPi (São Paulo)

Intentamos crear la Instancia A en Ohio con un grupo de seguridad nuevo, `G03-Simupi`, pero el lanzamiento volvió a fallar por el límite de vCPU. Como no podíamos bajar más el uso en esa región, la creamos en **São Paulo (sa-east-1)**. Los key pairs son por región, así que generamos uno nuevo, `G03-SaoPaulo`, y descargamos el `.pem`.

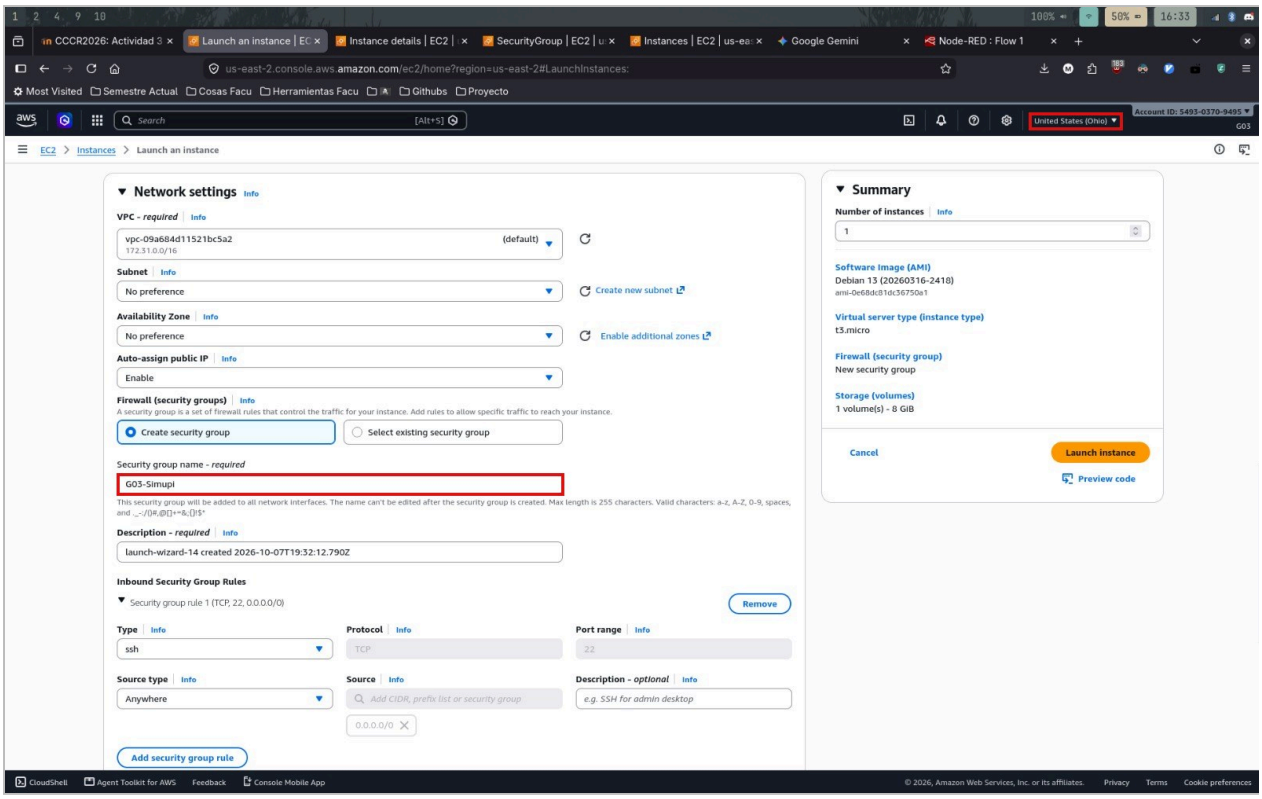


Figura 11: configuración de la Instancia A en Ohio con el grupo de seguridad G03-Simupi.

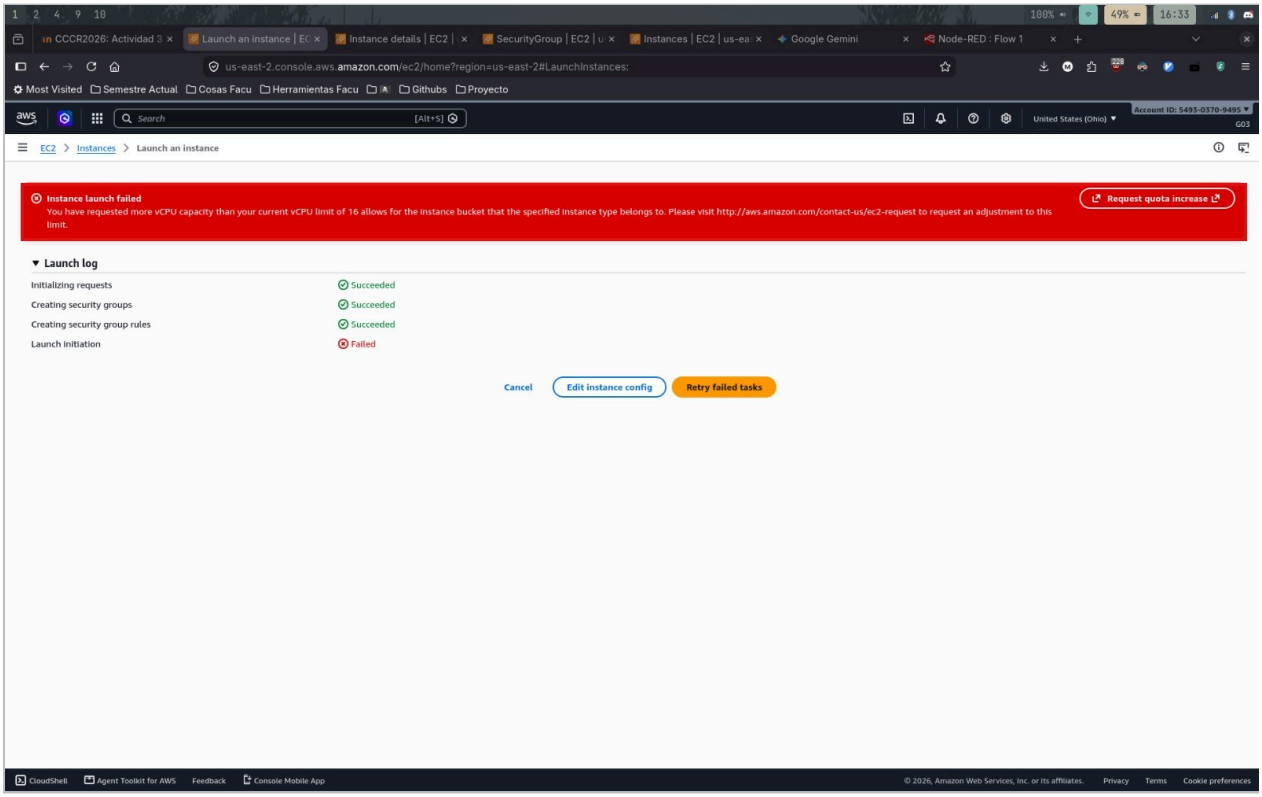


Figura 12: el lanzamiento en Ohio falla por el límite de 16 vCPU.

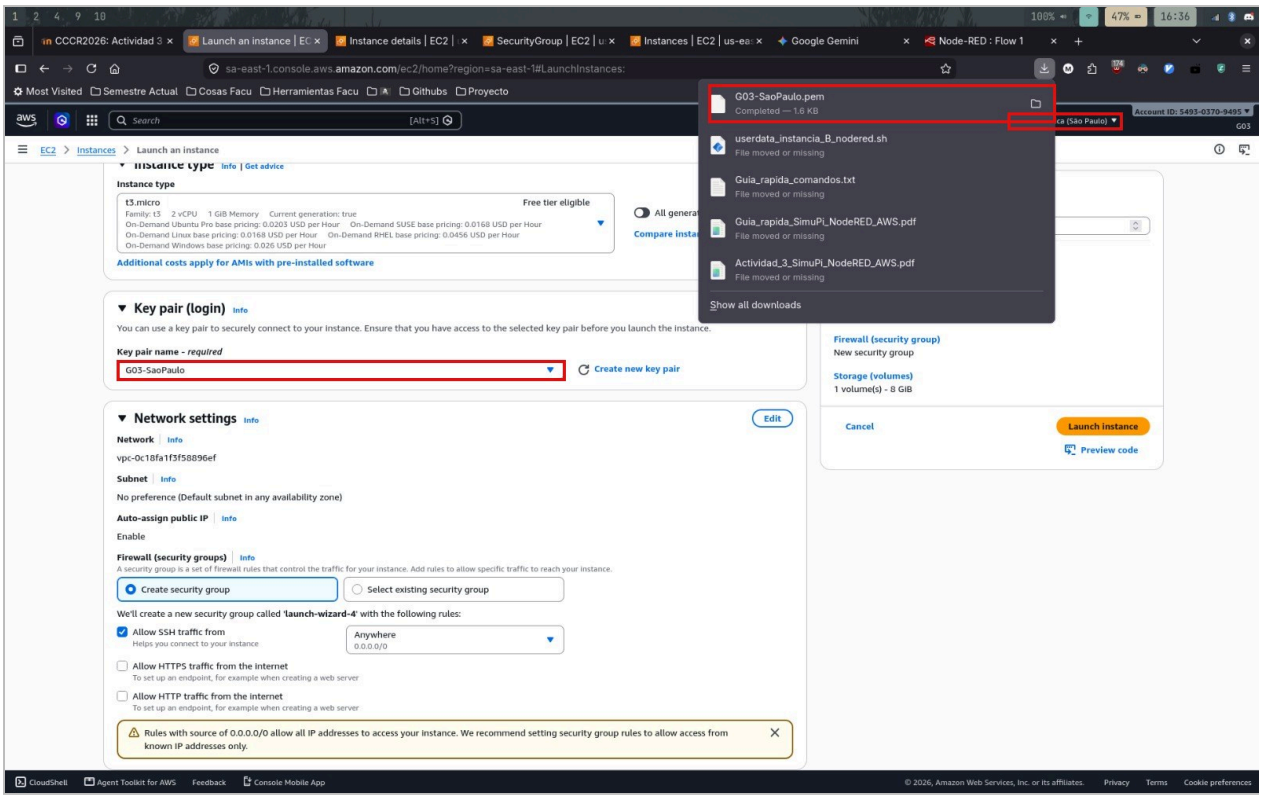


Figura 13: lanzamiento en São Paulo con el key pair nuevo G03-SaoPaulo.

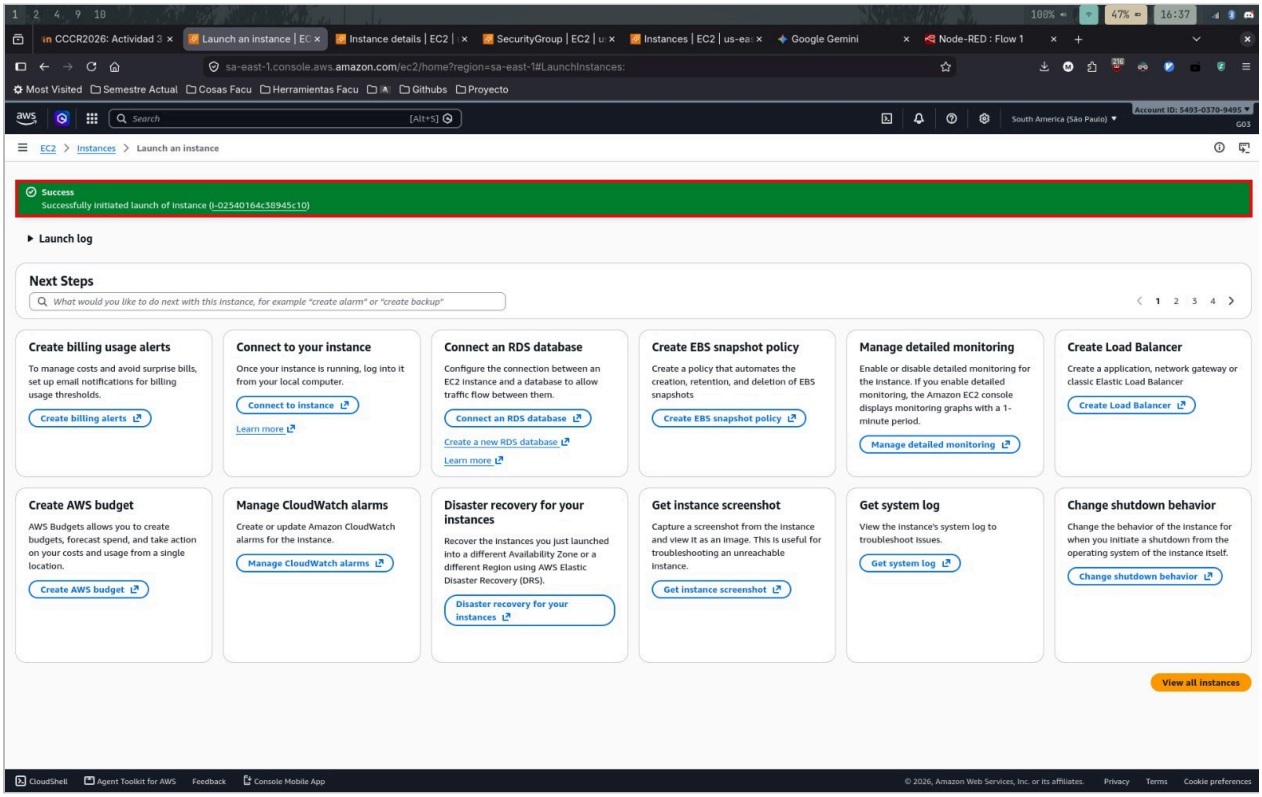


Figura 14: Instancia A lanzada en sa-east-1.

4) Grupo de seguridad de la Instancia A

La consigna pide que el puerto 1883 (MQTT) se permita solo desde el grupo de seguridad de B. Eso no es posible con las instancias en regiones distintas: un grupo de seguridad solo puede referenciar a

otro de la misma VPC, y además las instancias no comparten red privada. Lo consultamos y se nos indicó obviar ese punto, así que en su lugar permitimos el 1883 desde el bloque `3.129.25.0/24`, que contiene la IP pública de B. Una consecuencia es que, si la Instancia B se reinicia y cambia de IP pública, el bróker queda inaccesible hasta actualizar la regla.

Para el SSH (22) y la web/API de SimuPi (5000) usamos primero el bloque `/22` de la facultad y después los limitamos a nuestra IP exacta (`163.10.143.79/32`), como pide la consigna. Ninguno de los puertos 1883 y 5000 quedó abierto a `0.0.0.0/0`.

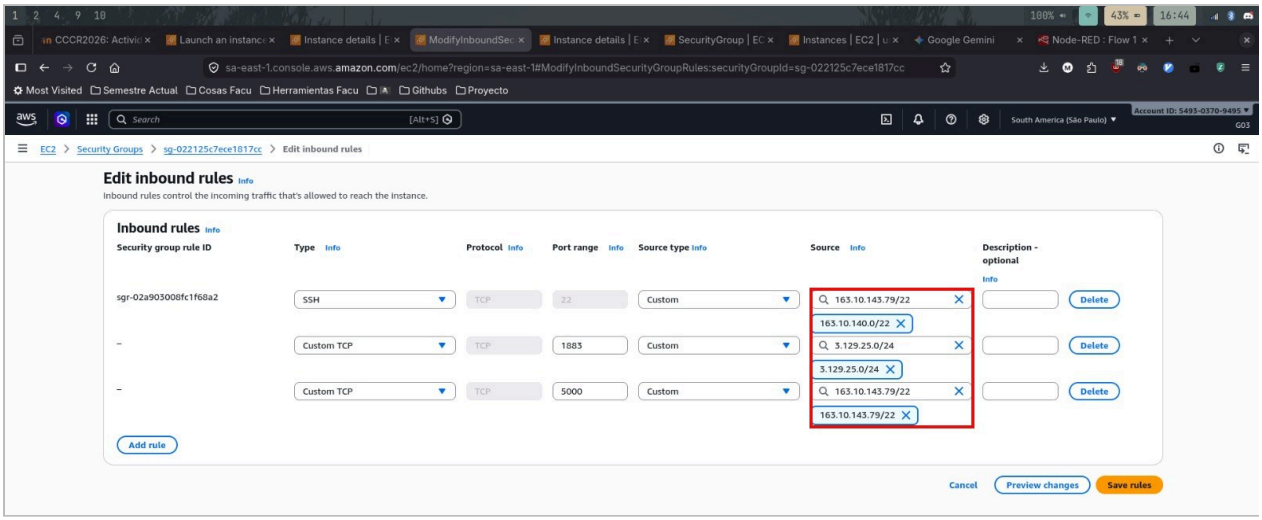


Figura 15: edición de las reglas de A: 1883 desde la red de B, 22 y 5000 desde la red de la facultad.

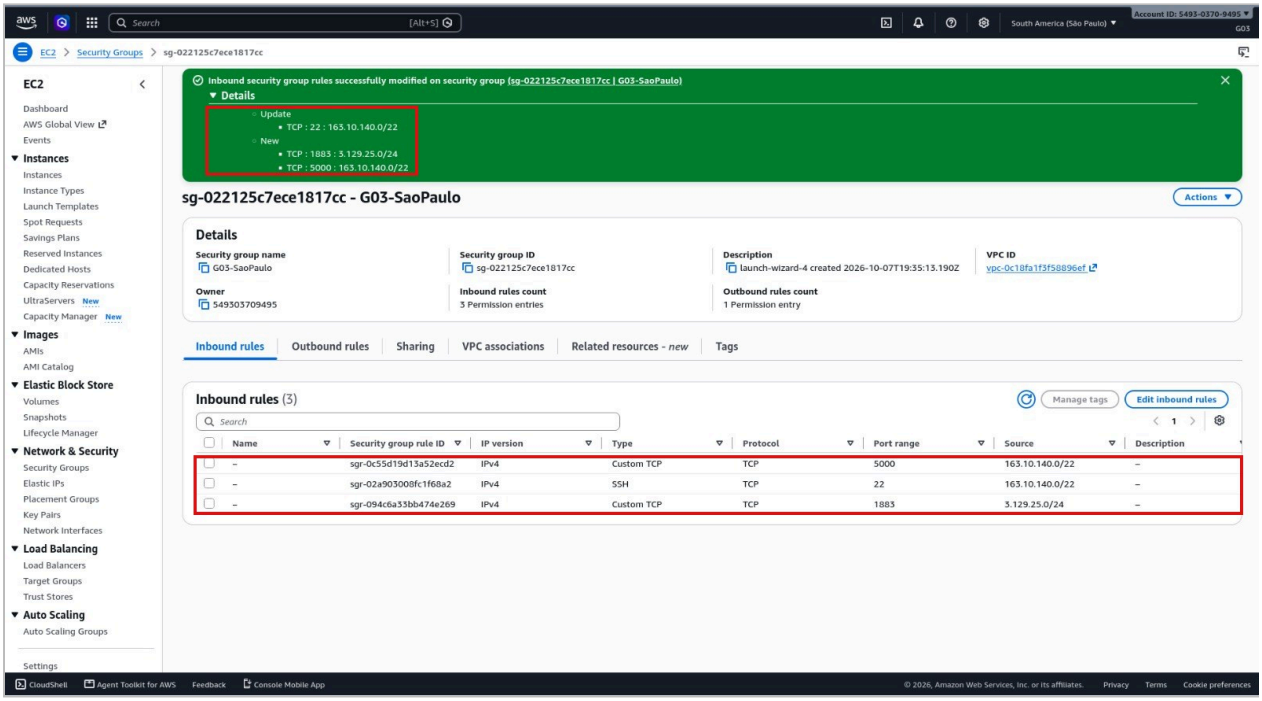


Figura 16: reglas de A guardadas en su primera versión.

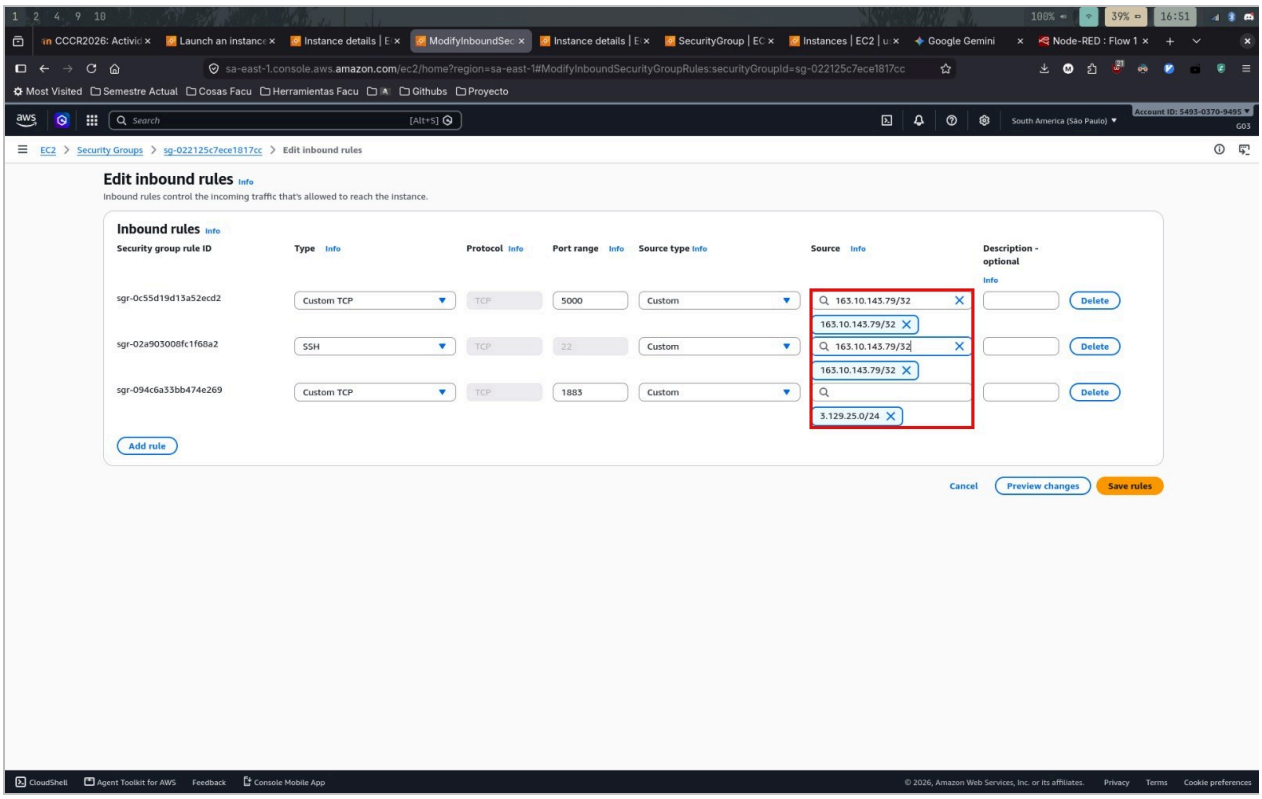


Figura 17: 22 y 5000 limitados a nuestra IP (/32).

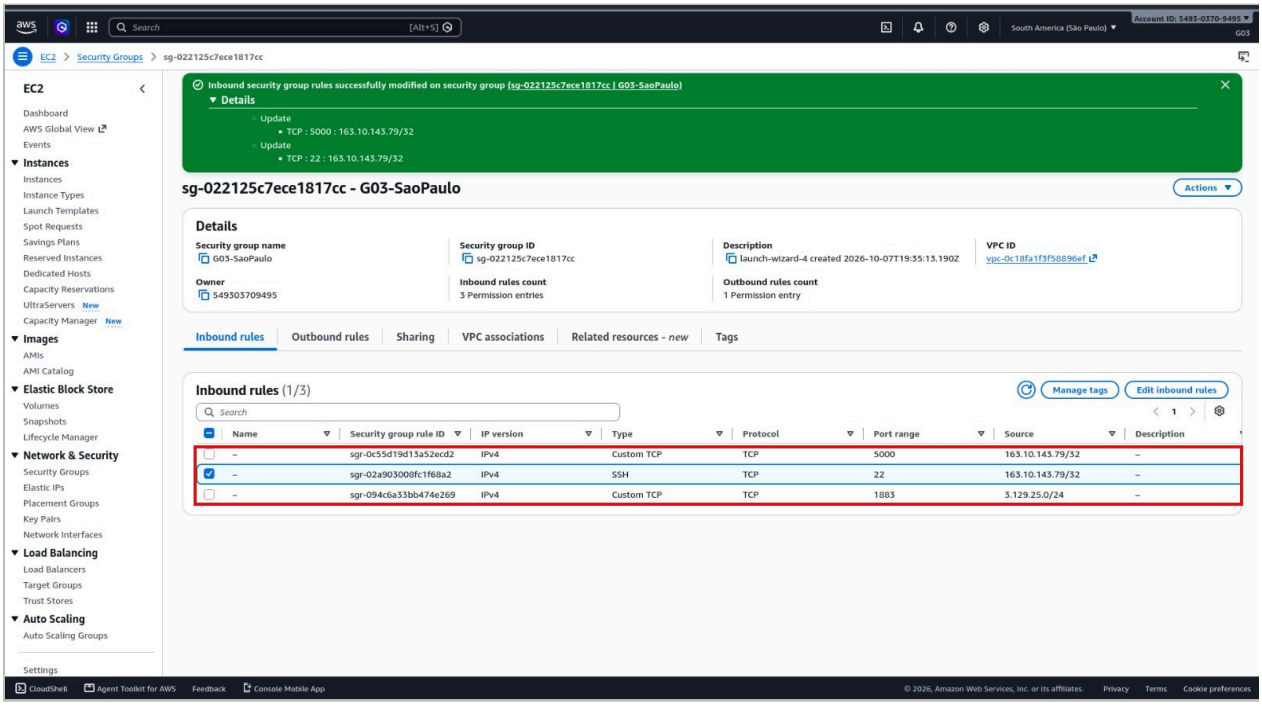


Figura 18: reglas finales de la Instancia A.

5) SimuPi en la Instancia A

Le dimos permisos al `.pem` y nos conectamos por SSH a la IP pública de A, `15.228.246.183`.

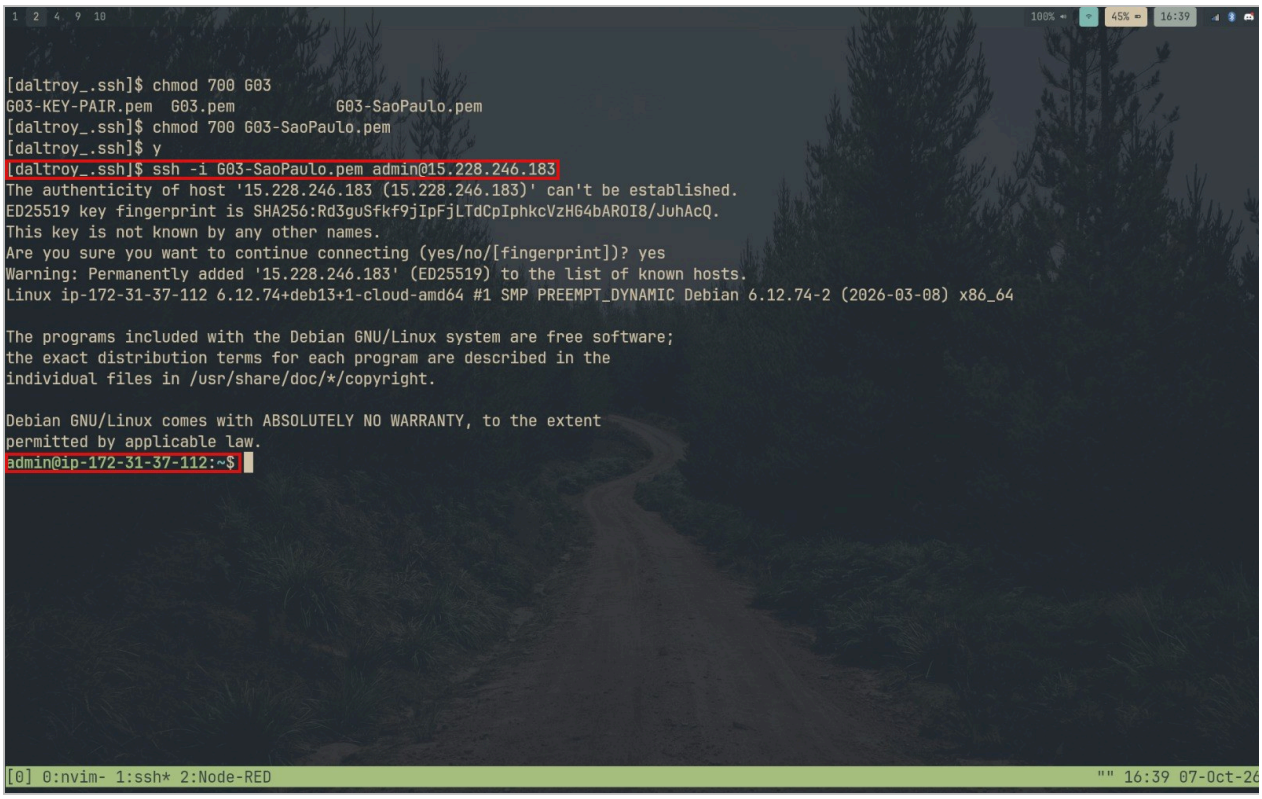


Figura 19: conexión SSH a la Instancia A con el key pair G03-SaoPaulo.

Descargamos el ejecutable de SimuPi para Linux desde el repositorio de la cátedra, lo dejamos corriendo en segundo plano con `nohup` (sigue vivo al cerrar la sesión SSH) y verificamos el servicio con `curl localhost:5000/api/health`, que respondió `{"status":"ok"}`.

```
wget -O simupi https://github.com/seguren/simupi/raw/main/bin/simupi-linux-amd64 &&
chmod +x simupi
nohup ./simupi > simupi.log 2>&1 &
curl localhost:5000/api/health
```

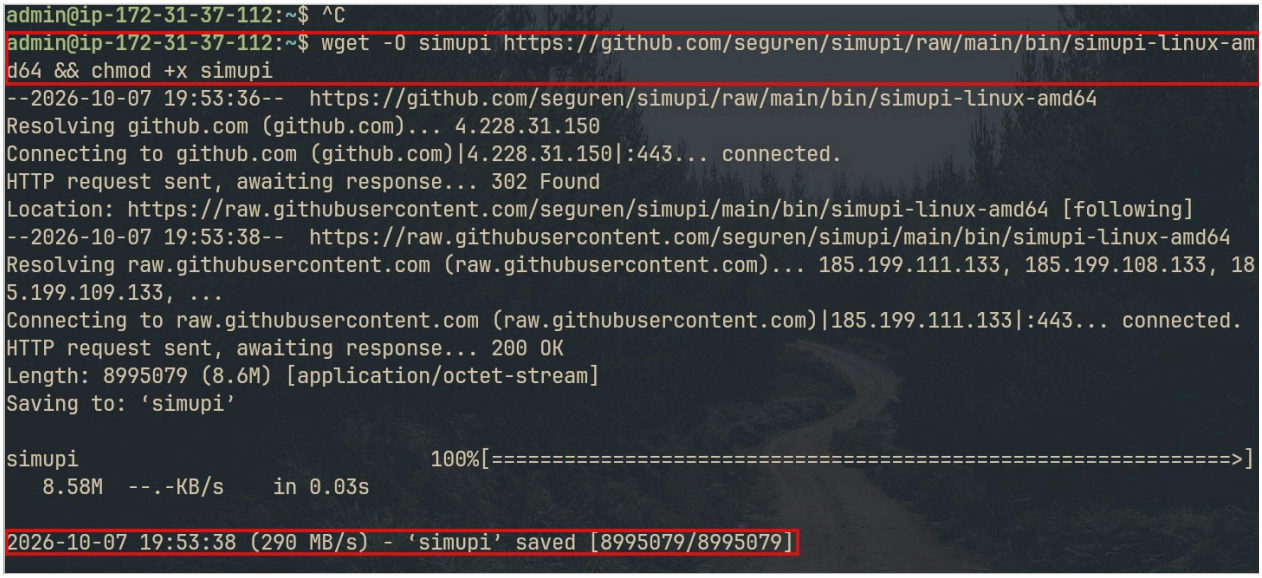


Figura 20: descarga del ejecutable de SimuPi.

```
admin@ip-172-31-37-112:~$ nohup ./simupi  >  simupi.log  2>&1  &
[1] 822
admin@ip-172-31-37-112:~$ curl localhost:5000/api/health
{"status":"ok"}
```

Figura 21: SimuPi corriendo con nohup y respuesta del endpoint de salud.

Para abrir la interfaz web armamos un túnel SSH que lleva el puerto 5000 de la instancia al `localhost:5000` de nuestra computadora. La interfaz abrió con el escenario por defecto, `garage`.

```
[daltroy_~]$ ssh -i /home/daltroy/.ssh/G03-SaoPaulo.pem -L 5000:localhost:5000 admin@15.228.246.183
Linux ip-172-31-37-112 6.12.74+deb13+1-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08)
 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Oct  7 19:39:18 2026 from 163.10.143.79
admin@ip-172-31-37-112:~$
```

Figura 22: túnel SSH del puerto 5000 de la Instancia A.

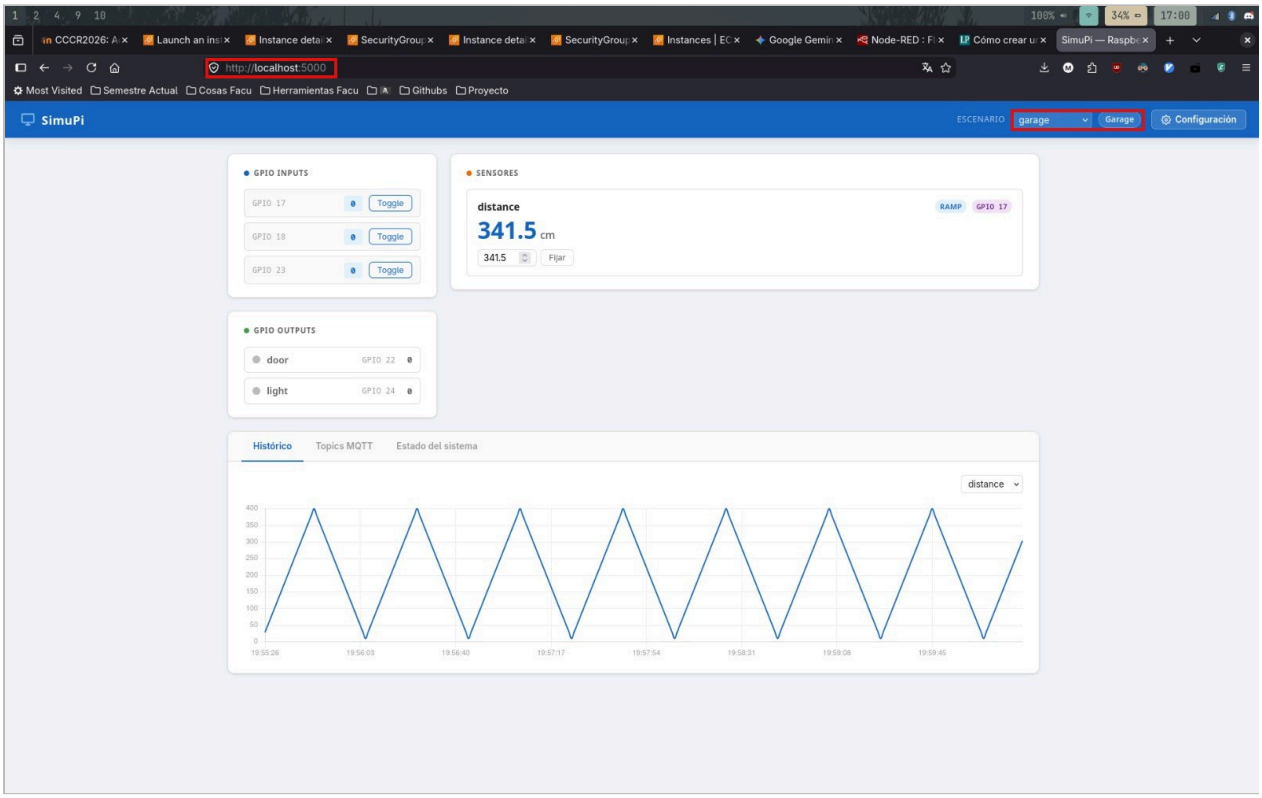


Figura 23: interfaz web de SimuPi en localhost:5000 con el escenario garage.

6) Dashboard en la Instancia B

El paquete `@flowfuse/node-red-dashboard` quedó instalado por el script de user data. Lo verificamos en *User Settings → Palette*: versión **1.33.0**, sobre Node-RED 5.0.7.

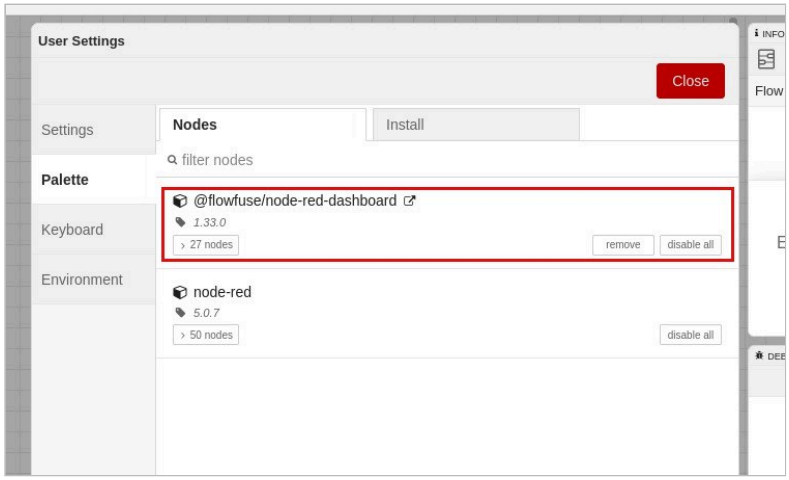


Figura 24: @flowfuse/node-red-dashboard instalado en la Instancia B.

7) Conexión entre B y A

En Node-RED configuramos el bróker MQTT. Como A y B están en regiones distintas no podíamos usar la IP privada de A (como indica la consigna), así que usamos su IP pública: `15.228.246.183:1883`. Con un nodo *mqtt in* suscripto a `sensor/distance` y un nodo *debug* comprobamos que llegan las lecturas del sensor del escenario garage, que bajan en escalones como corresponde al modo `ramp`.

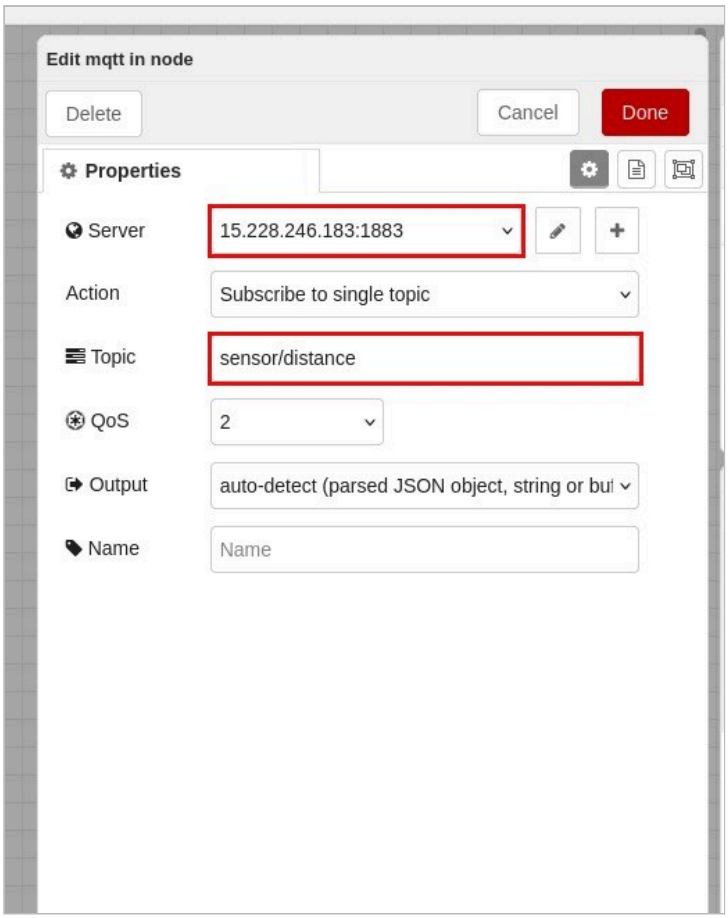


Figura 25: nodo mqtt in con el bróker de SimuPi (IP pública de A, puerto 1883).

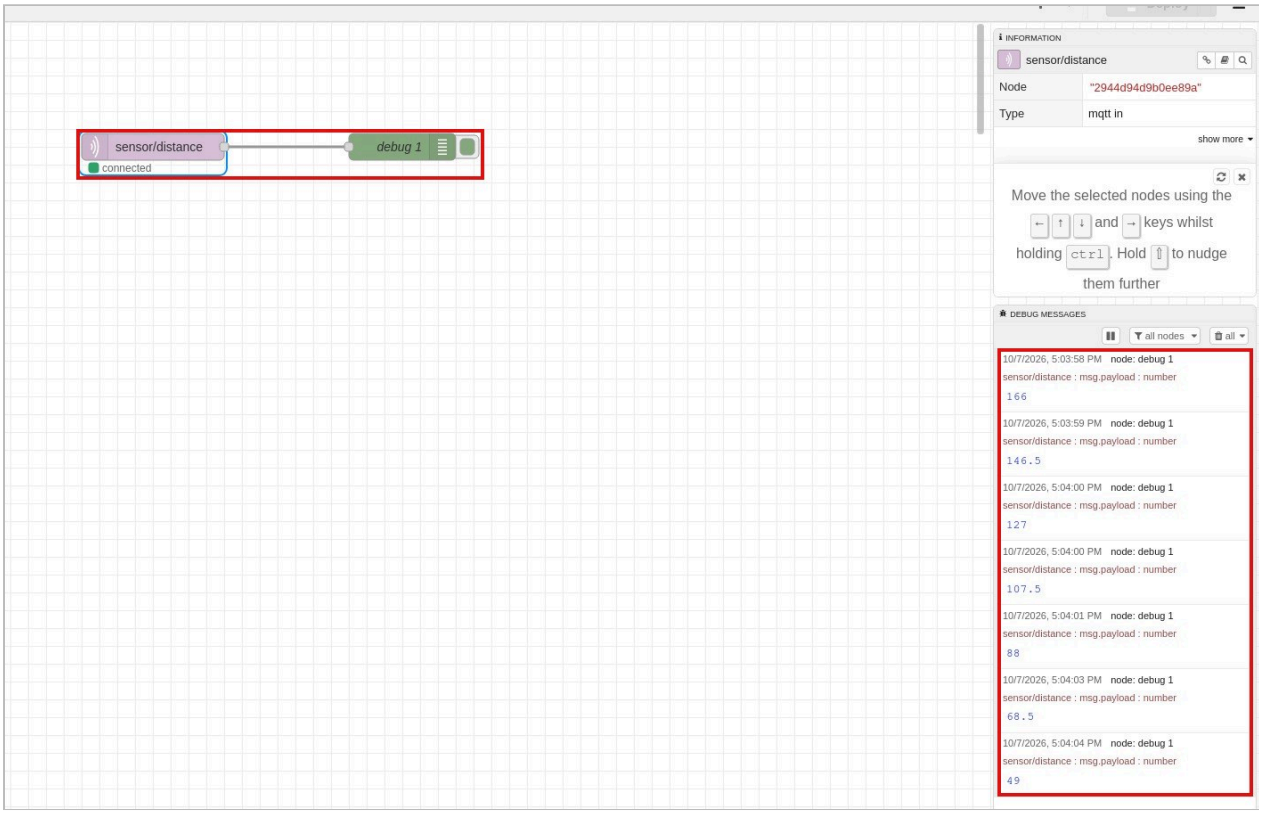


Figura 26: lecturas de sensor/distance en el panel de debug.

Hicimos lo mismo con la entrada `gpio/in/17`: al presionar *Toggle* en SimuPi llegan los valores 1 y 0.

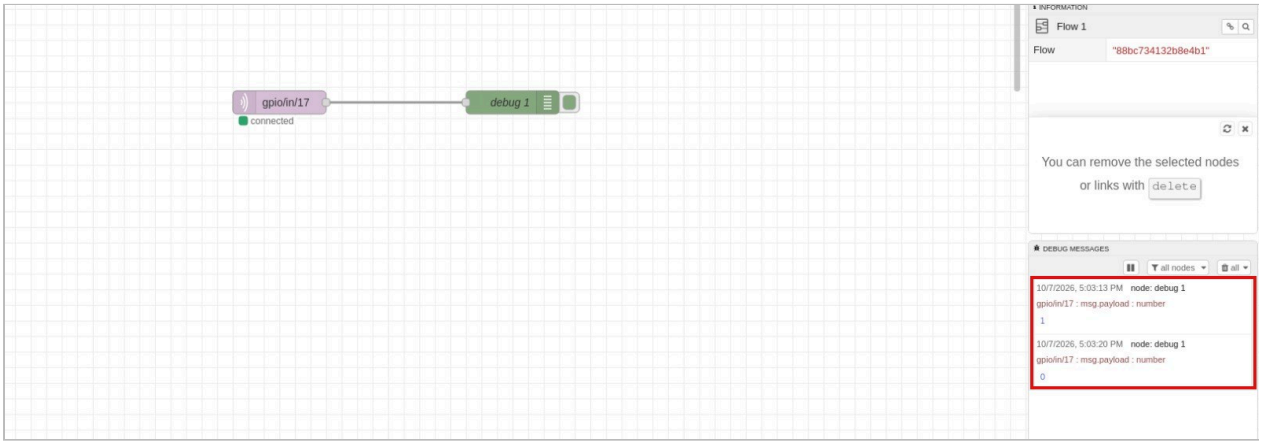


Figura 27: cambios de gpio/in/17 recibidos en Node-RED.

Parte 2 – Dashboard con los escenarios preconfigurados

1) Entradas GPIO

Empezamos por las entradas: un *mqtt in* por pin (`gpio/in/17` y `gpio/in/18`) conectado a un nodo *text* del dashboard, con un debug para verificar los mensajes.

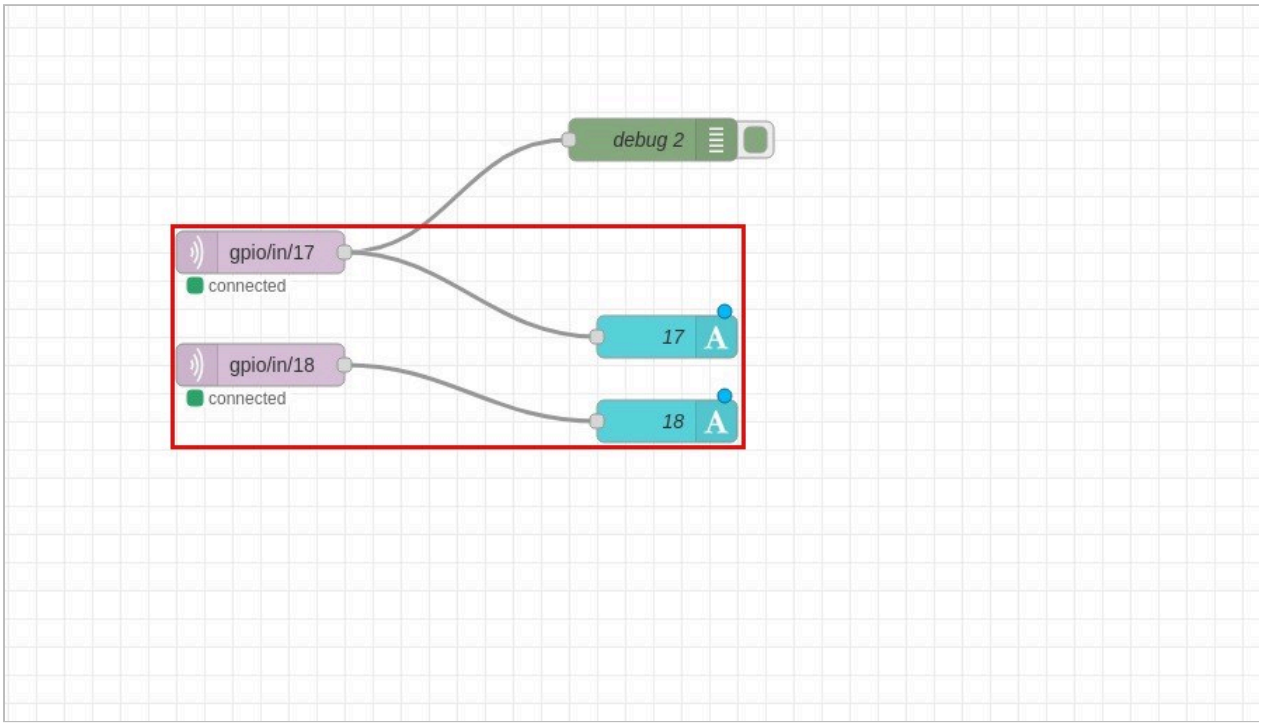


Figura 28: flujo de las entradas GPIO 17 y 18.

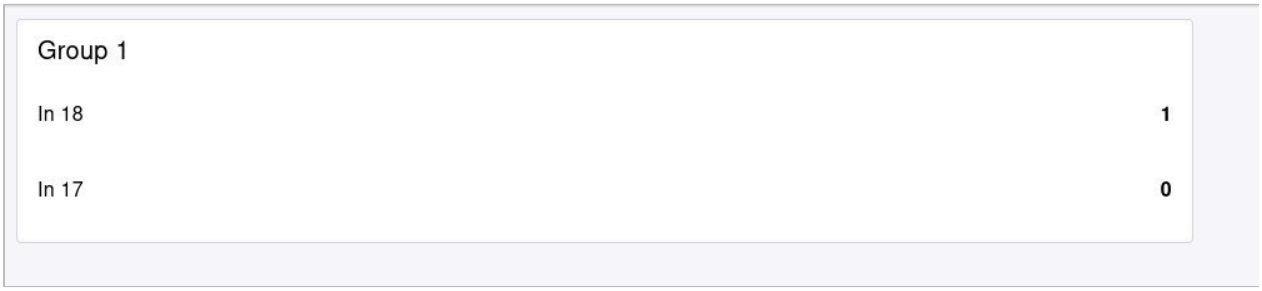


Figura 29: primer dashboard con el estado de las entradas.

2) Dashboard del escenario greenhouse

Elegimos el escenario `greenhouse` (temperatura en modo `wave` entre 15 y 40 °C, humedad en modo `walk` entre 30 y 90 %, ventilador en el GPIO 22 y bomba en el GPIO 24). El flujo tiene tres partes:

- **Sensores:** `sensor/temperature` y `sensor/humidity` van cada uno a un gauge (valor actual) y a un chart (evolución en el tiempo).
- **Actuadores:** los switches *Fan* y *Pump* publican `"1"`/`"0"` en `gpio/out/22` y `gpio/out/24`.
- **Estado:** las entradas `gpio/in/17` y `gpio/in/18` se muestran como texto. Como SimuPi no publica el estado de las salidas, nos suscribimos también a `gpio/out/22` y `gpio/out/24`: así el dashboard

muestra el último comando que pasó por el bróker, venga de este dashboard o de otro cliente.

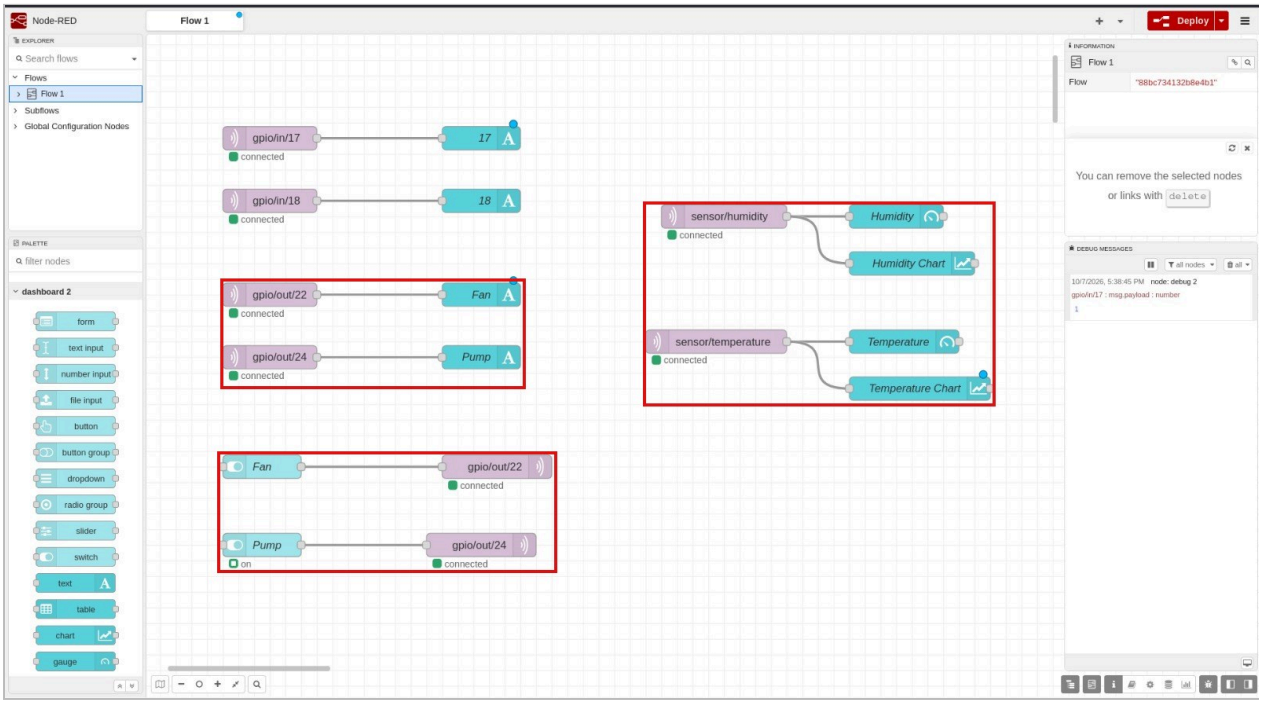


Figura 30: flujo del escenario greenhouse: estado de salidas, switches de control y sensores.



Figura 31: dashboard de greenhouse: estado del ventilador (0) y de la bomba (1), gráficos y controles.

3) Versión final

En la versión final reemplazamos los textos 0/1 de las salidas por switches de estado. Una función *1=True / 0=False Mapper* convierte el payload de `gpio/out/<pin>` en booleano, y los switches *Estado Ventilador* y *Estado Bomba* lo muestran encendido o apagado.

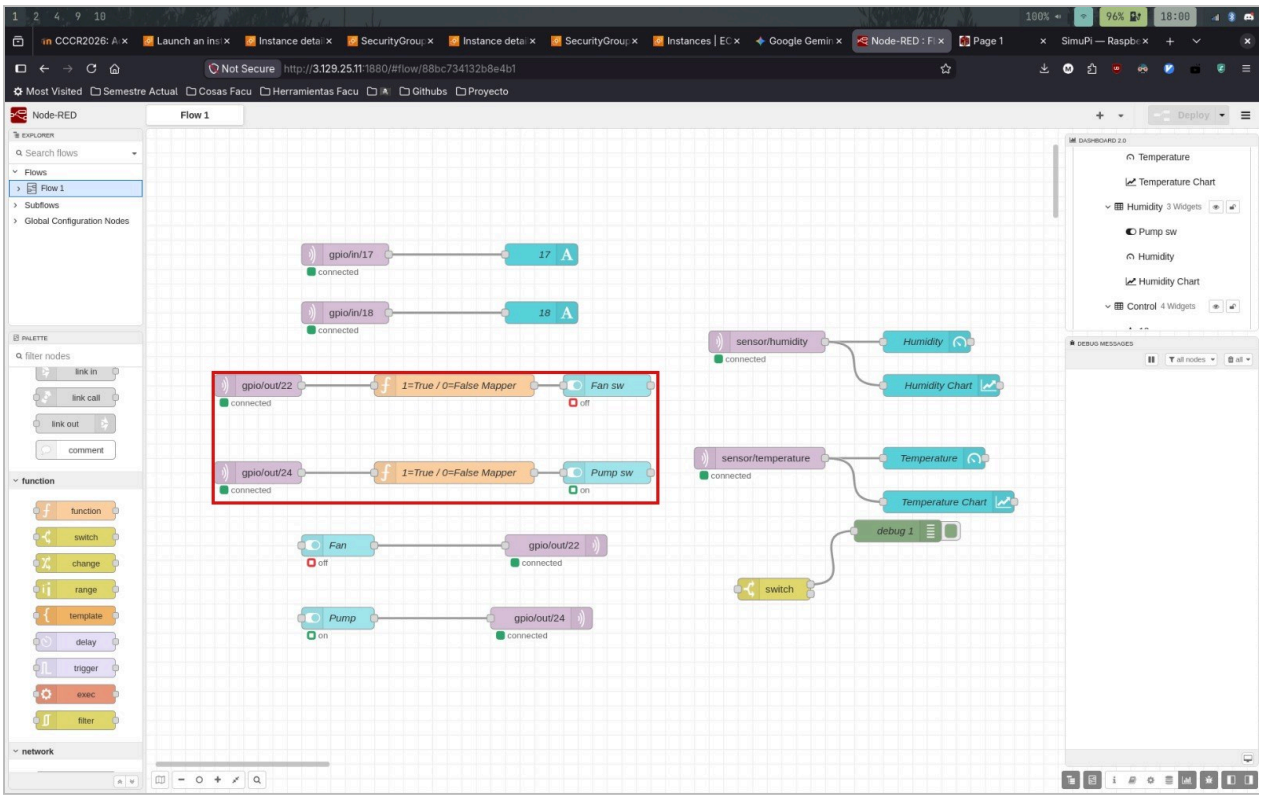


Figura 32: flujo final: gpio/out/22 y gpio/out/24 pasan por el mapper hacia los switches de estado.

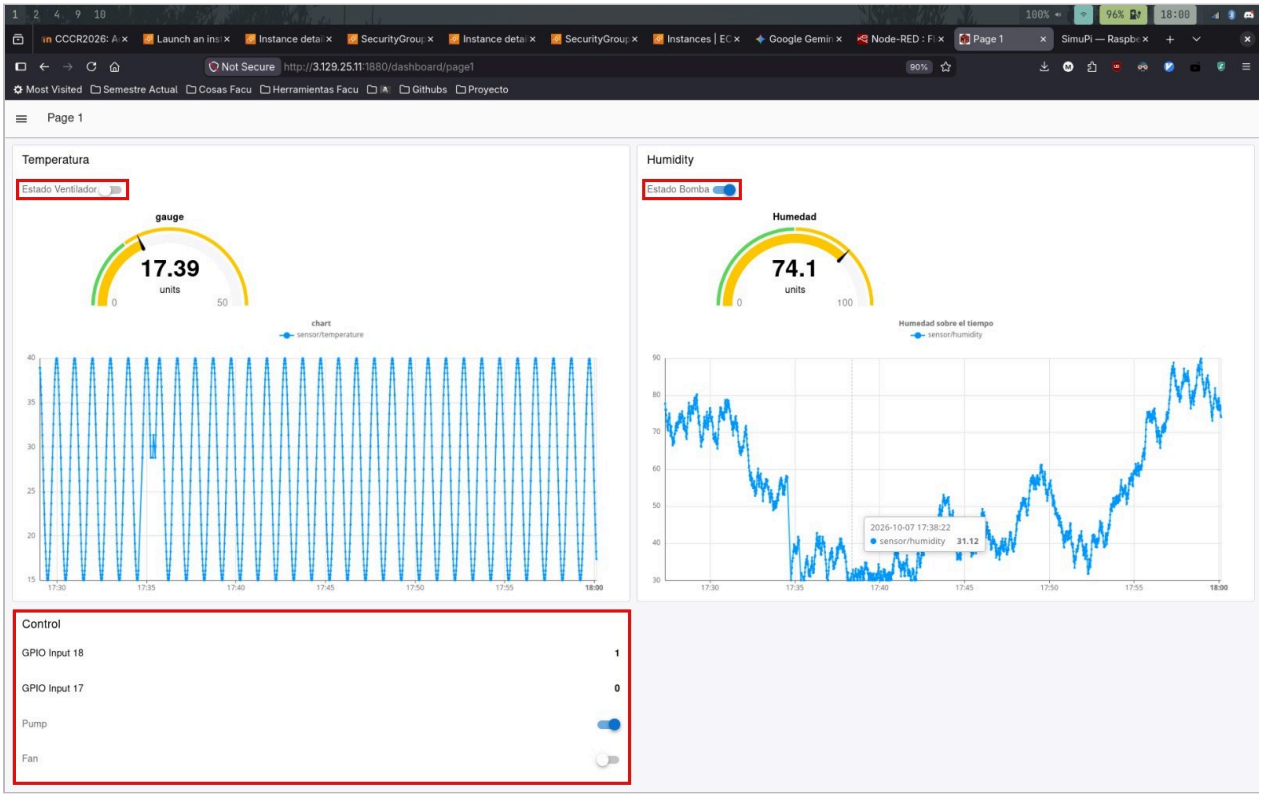


Figura 33: dashboard final de greenhouse con el estado del ventilador y de la bomba.

> El punto 7 (adaptar el dashboard a un segundo escenario) no se realizó: la cátedra nos indicó obviarlo por su complejidad para esta práctica.

Parte 3 – Escenario propio: sala de servidores

1) Caso de uso

El escenario `sala_servidores` simula el monitoreo y control de una sala de servidores pequeña, donde la temperatura, la humedad y el consumo eléctrico deben mantenerse en rangos seguros para evitar fallas del hardware. El sistema mide esas tres variables, detecta la apertura de la puerta de la sala y actúa sobre dos salidas: un equipo de refrigeración adicional y una baliza de alarma. Los rangos siguen la recomendación habitual para salas de cómputo: temperatura de operación hasta 27 °C (alarma sobre 30 °C) y humedad relativa entre 40 % y 60 %.

El dashboard de Node-RED (`/dashboard/sala-servidores`) permite al operador ver los valores en tiempo real y su histórico, elegir entre modo MANUAL y AUTOMÁTICO, comandar las salidas a mano y ver las alarmas activas con su causa.

2) Entorno de ejecución

Como las instancias EC2 de las Partes 1 y 2 se terminaron al finalizar la clase, como pide la consigna, desarrollamos la Parte 3 con Docker en una computadora del grupo. Cada instancia se reemplazó por un contenedor, ambos definidos en un mismo `docker-compose.yml`:

Contenedor	Reemplaza a	Imagen	Puertos publicados
`simupi`	Instancia A	compilada desde el repositorio de SimuPi (Go) sobre `distroless/static`, sin usuario root	5000 (web/API), solo en `127.0.0.1`
`nodered`	Instancia B	`nodered/node-red:4.0` + `@flowfuse/node-red-dashboard`	1880 (editor y dashboard), solo en `127.0.0.1`

```
services:
  simupi:
    build: .
    ports:
      - "127.0.0.1:5000:5000"   # Web UI + API REST, solo local
    # 1883 (MQTT) no se publica: Node-RED lo alcanza por la red interna como simupi:1883
    volumes:
      - simupi-data:/app/data
  nodered:
    image: nodered/node-red:4.0
    ports:
      - "127.0.0.1:1880:1880"
    volumes:
      - nodered-data:/data
```

Los contenedores comparten la red interna que crea Docker Compose, así que Node-RED llega al bróker por nombre, `simupi:1883`. El 1883 no se publica fuera de esa red y los puertos 5000 y 1880 quedan atados a `127.0.0.1`. Es el mismo criterio que los grupos de seguridad de AWS: el bróker sin usuario ni contraseña solo es accesible para Node-RED, y las interfaces web solo desde nuestra máquina.

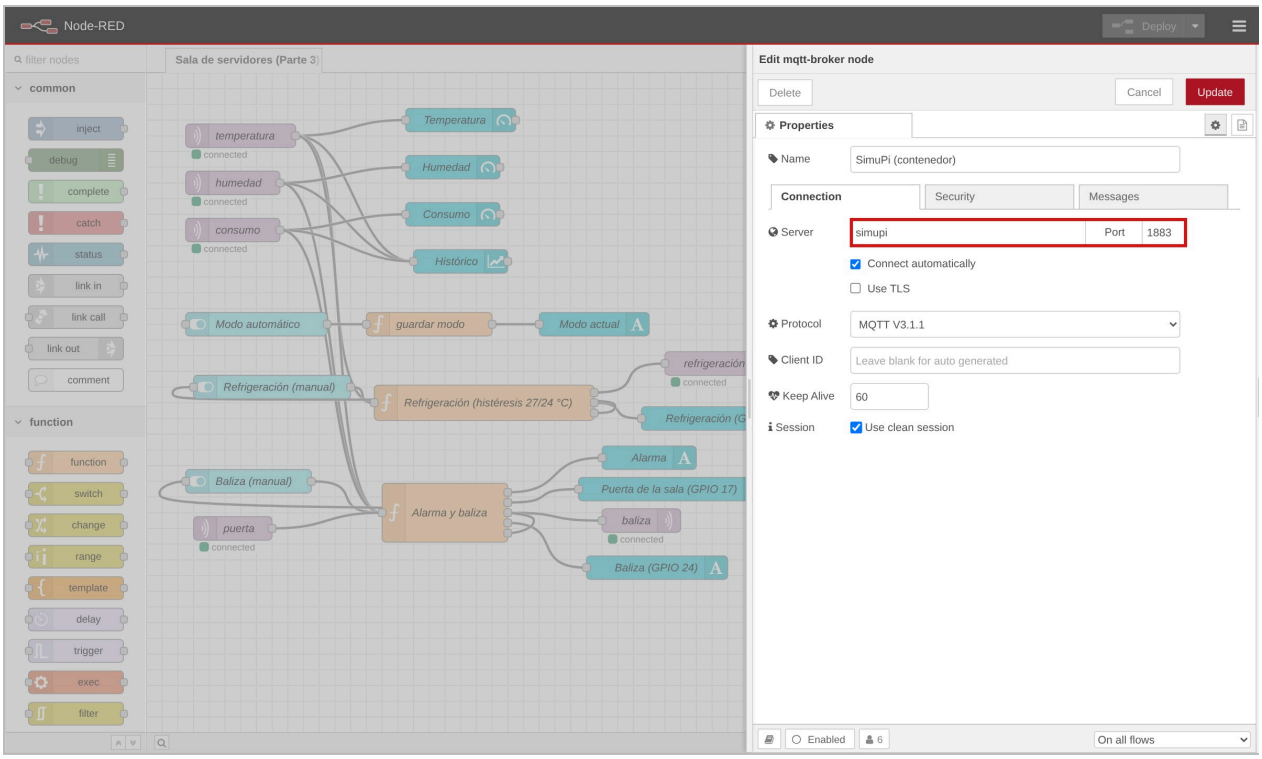


Figura 34: bróker de Node-RED apuntando al contenedor de SimuPi por su nombre en la red de Docker.

3) Sensores

El escenario tiene tres sensores con dos modos de simulación: `walk` (caminata aleatoria) para la temperatura y la humedad, que varían lentamente, y `triangle` para el consumo, que reproduce las rampas de carga eléctrica de los racks.

Sensor	Tópico MQTT	Modo	Rango simulado	Unidad	Intervalo	Parámetro	Rango seguro
temperatura	`sensor/temperatura`	walk	18 – 34	°C	1 s	step = 0,8 °C	≤ 30 °C
humedad	`sensor/humedad`	walk	30 – 70	%	2 s	step = 1,5 %	40 – 60 %
consumo	`sensor/consumo`	triangle	4 – 12	kW	2 s	period = 180 s	≤ 10,5 kW

4) Pines GPIO y tópicos MQTT

Pin	Tipo	Alias	Tópico MQTT	Sentido	Valores
17	GPIO input	`puerta_sala`	`gpio/in/17`	SimuPi → Node-RED	1 = abierta, 0 = cerrada
22	GPIO output	`refrigeracion`	`gpio/out/22`	Node-RED → SimuPi	1 = encendida, 0 = apagada
24	GPIO output	`baliza_alarma`	`gpio/out/24`	Node-RED → SimuPi	1 = activa, 0 = apagada

5) Creación del escenario en SimuPi

Escribimos la definición en `sala_servidores.json` (se entrega junto con el informe) y la guardamos con la misma operación que usa el botón *Guardar como escenario* de la ventana Configuración:

```
curl -X POST localhost:5000/api/config/save/sala_servidores \
     -H "Content-Type: application/json" -d @sala_servidores.json
```

El escenario aparece en el selector, y al cargarlo SimuPi muestra la entrada `puerta_sala`, las salidas `refrigeracion` y `baliza_alarma` y los tres sensores con su modo.

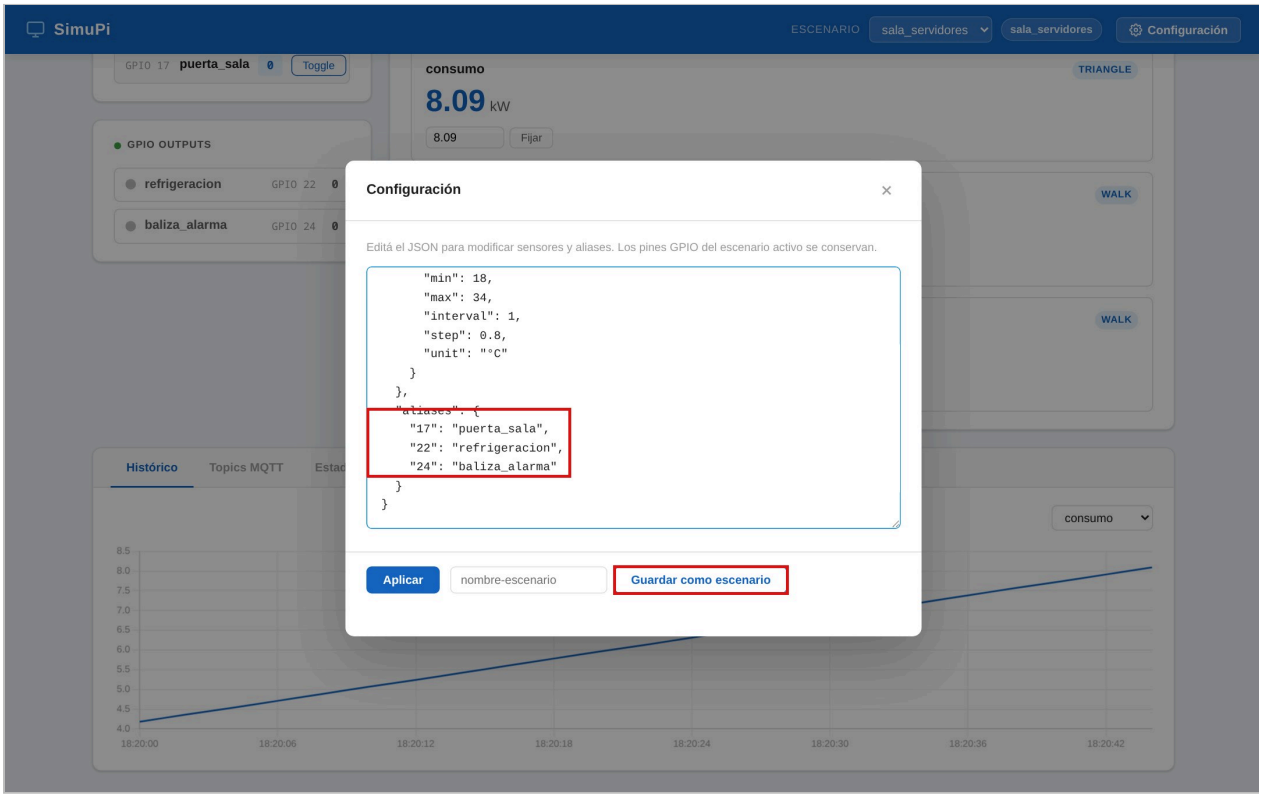


Figura 35: ventana Configuración de SimuPi con los alias del escenario y el botón Guardar como escenario.

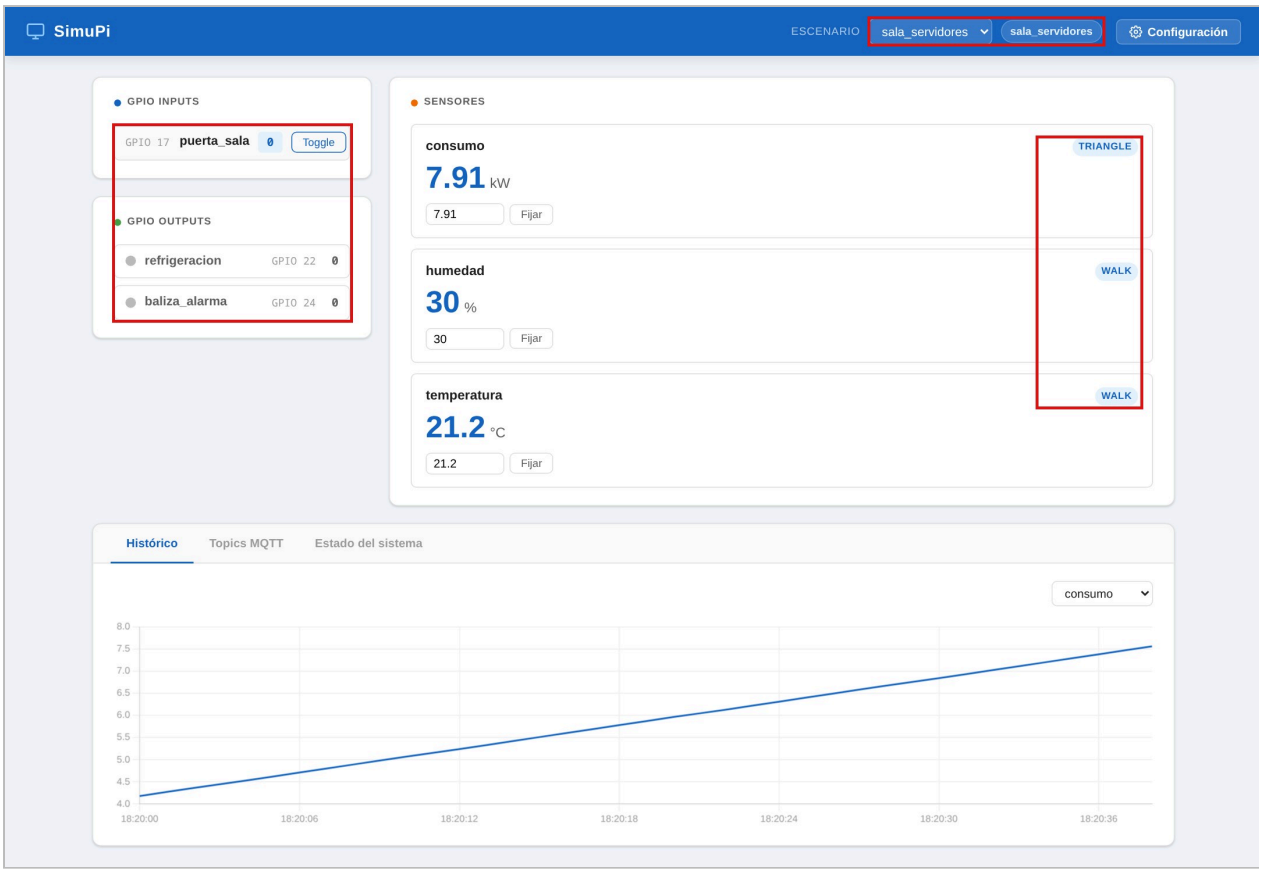


Figura 36: escenario sala_servidores cargado: selector, pines con sus alias y modos de cada sensor.

6) Flujo de Node-RED

El flujo tiene 21 nodos. Los tres *mqtt in* de sensores alimentan los gauges y el gráfico histórico, y además entran a la lógica. Toda la lógica está en tres nodos function: *guardar modo*, *Refrigeración (histéresis 27/24 °C)* y *Alarma y baliza*. Cada actuador recibe en la misma función su switch manual y los sensores, y el modo decide cuál de los dos manda. El estado se guarda en el contexto del flujo (`modo_auto`, `refrig`, `baliza`, `valores`).

Cada función devuelve también el estado real de la salida al switch manual correspondiente. Así, si el modo automático enciende la refrigeración y el operador pasa a MANUAL, el switch ya aparece encendido; y si en AUTOMÁTICO se toca un switch, vuelve solo a su posición.

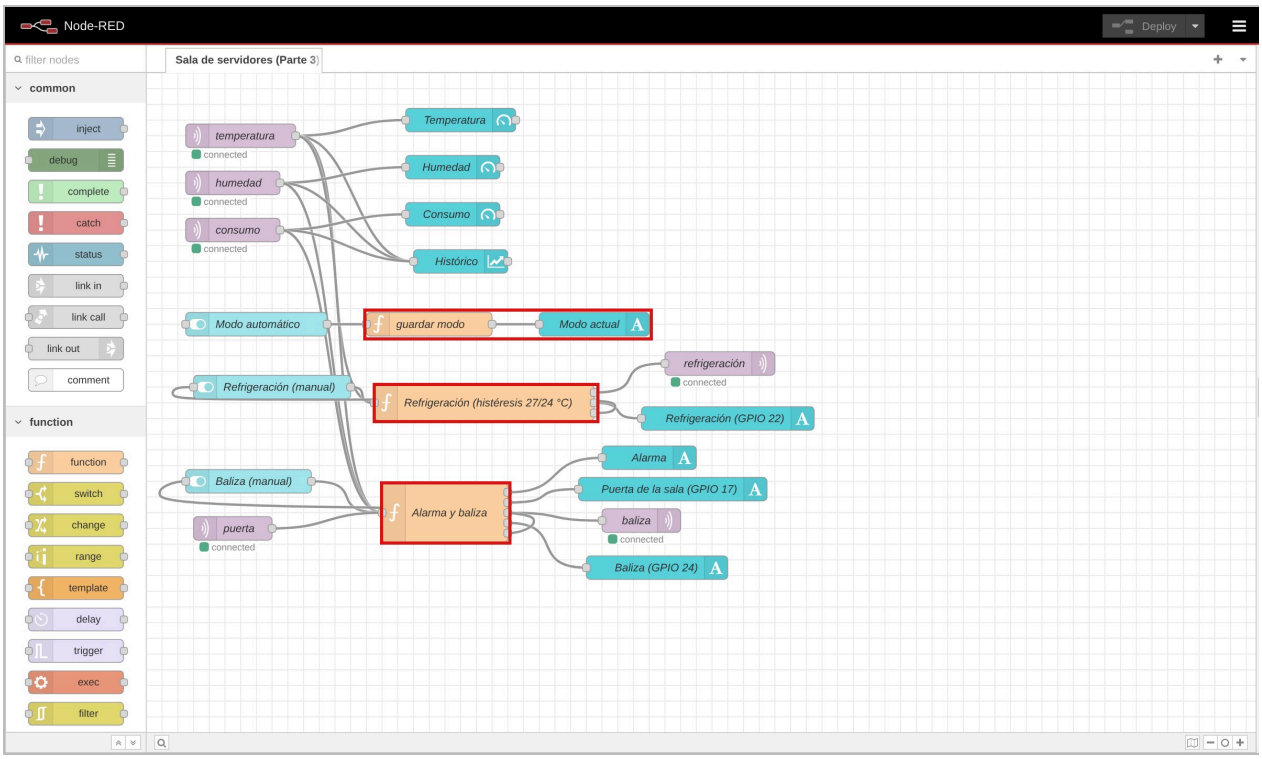


Figura 37: flujo de la sala de servidores; resaltadas las tres funciones con la lógica.

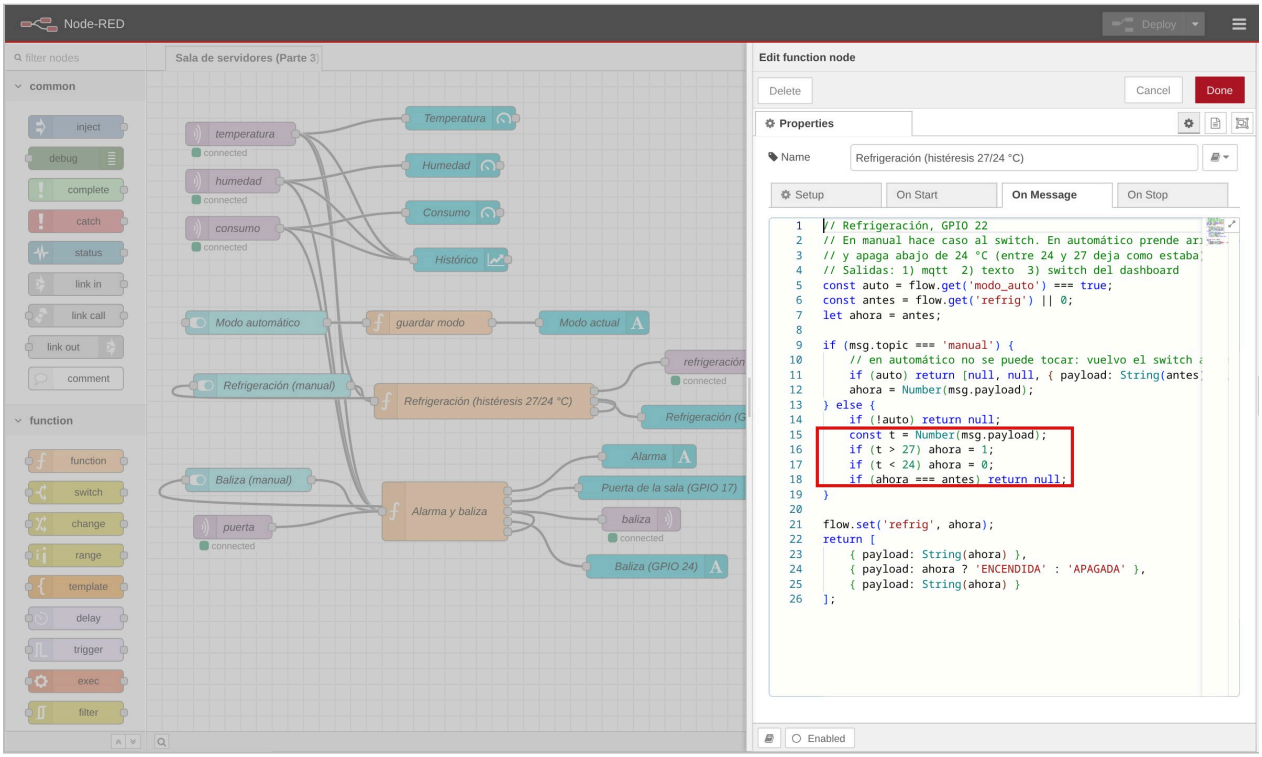


Figura 38: función de refrigeración: enciende sobre 27 °C, apaga bajo 24 °C y publica solo si el estado cambia.

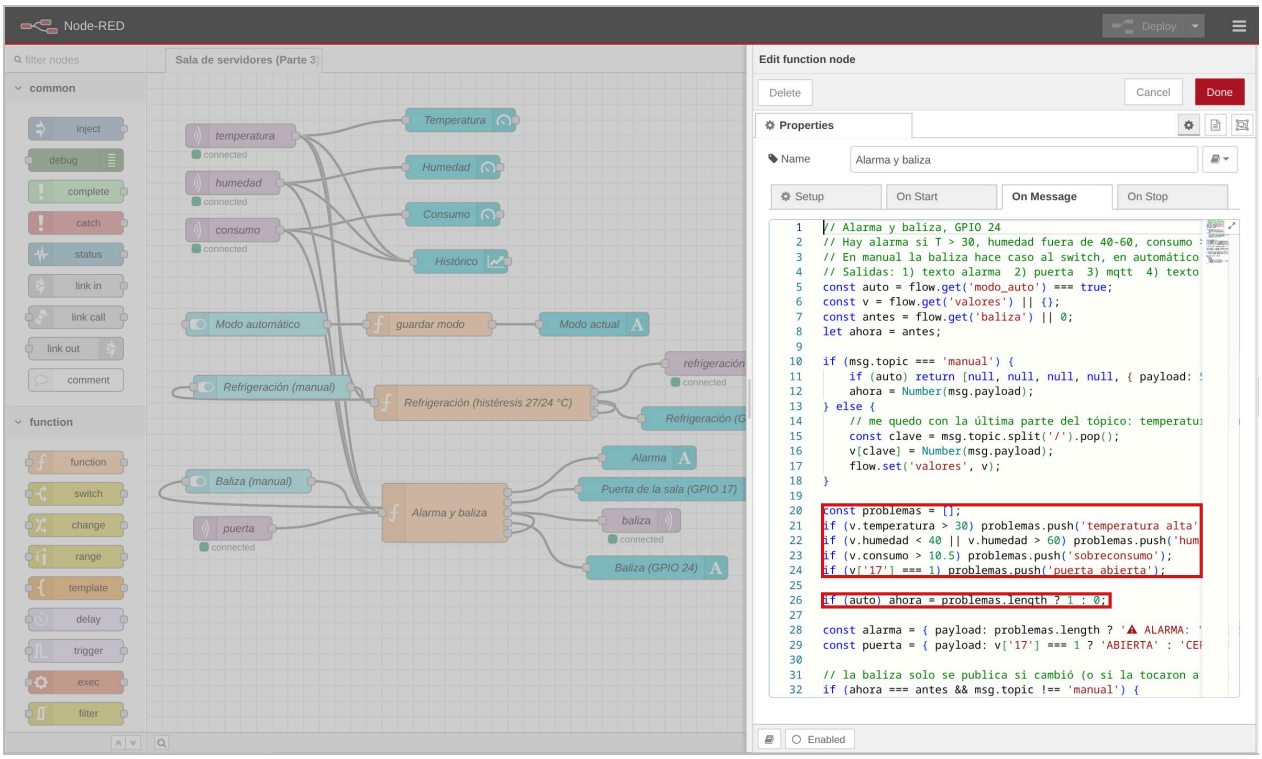


Figura 39: función de alarma: condiciones fuera de rango y baliza siguiendo a la alarma en modo automático.

7) Reglas de automatización

Regla	Condición	Acción	Modo
Refrigeración con histéresis	T > 27 °C enciende; T < 24 °C apaga; entre 24 y 27 °C mantiene el estado	`gpio/out/22` = 1 / 0	Solo AUTOMÁTICO
Alarma fuera de rango	T > 30 °C, humedad < 40 % o > 60 %, consumo > 10,5 kW, o puerta abierta	Texto de alarma con la causa; `gpio/out/24` = 1 mientras dure	Texto: siempre. Baliza: solo AUTOMÁTICO
Modo MANUAL / AUTOMÁTICO	Interruptor “Modo automático” del dashboard	Habilita la lógica automática o los switches manuales	—
Control manual	Switches “Refrigeración (manual)” y “Baliza (manual)”	Publican 1 / 0 en `gpio/out/22` y `gpio/out/24`	Solo MANUAL

La banda de histéresis de 3 °C evita que la refrigeración arranque y pare en cada lectura cuando la temperatura oscila alrededor de un único umbral. Las salidas se publican solo cuando su estado cambia.

8) Dashboard

El dashboard tiene tres grupos: *Condiciones de la sala* (gauges con franjas verde, amarilla y roja según el rango seguro de cada variable), *Control y alarmas* (modo, switches manuales, estado de las salidas, alarma y puerta) e *Histórico* (las tres series en un gráfico). En la figura, en modo manual y con

los valores de la simulación, la humedad está por debajo de 40 % y el dashboard lo informa como alarma.

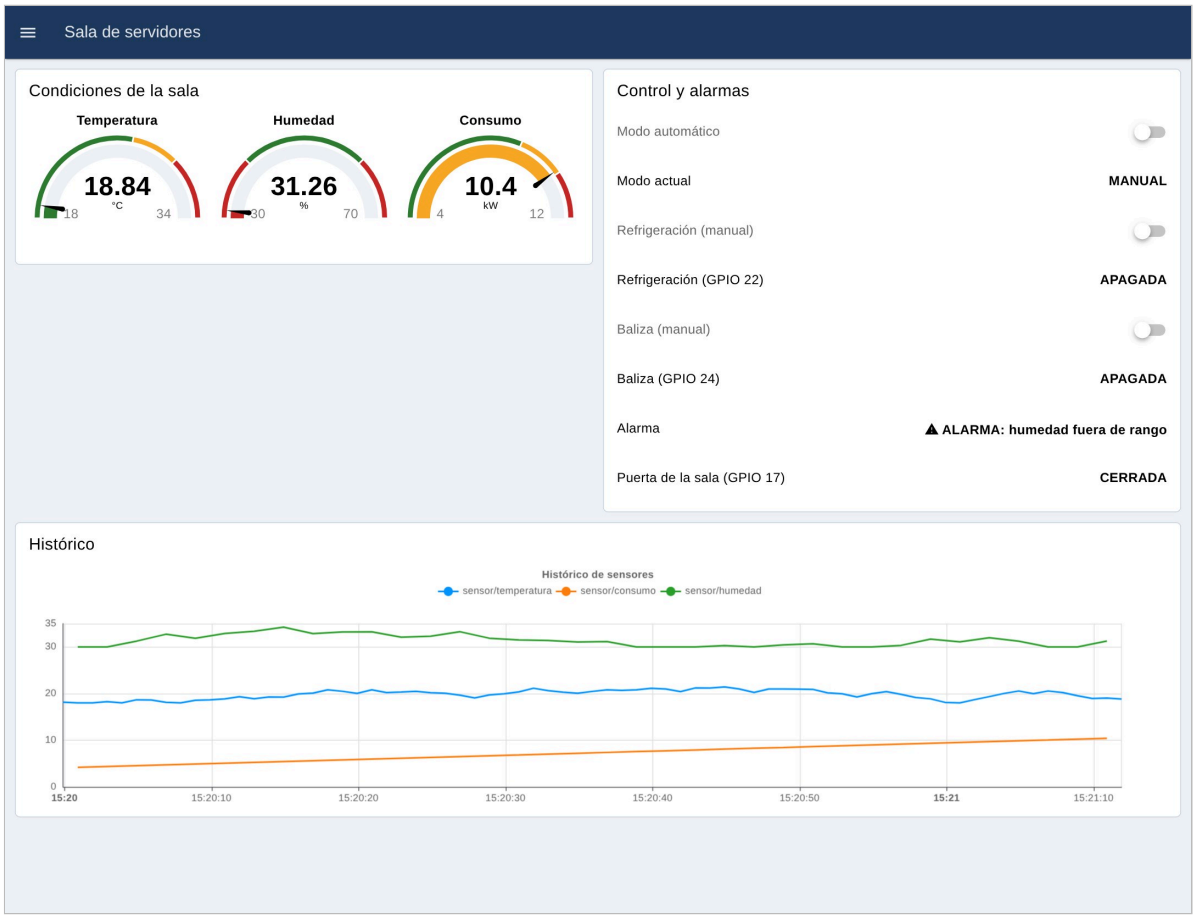


Figura 40: dashboard de la sala de servidores con los valores de la simulación.

9) Pruebas

Para no depender del ciclo de la simulación forzamos cada caso con la API de SimuPi. Antes de empezar fijamos la humedad en 50 % y el consumo en 7 kW, para que solo cambie la variable de cada prueba. En cada caso mostramos el dashboard y, a la derecha, la interfaz de SimuPi, donde se ve el estado real de los pines.

a) Control manual

En modo MANUAL encendimos el switch *Refrigeración (manual)*: el dashboard informa ENCENDIDA y SimuPi pone el GPIO 22 en 1.

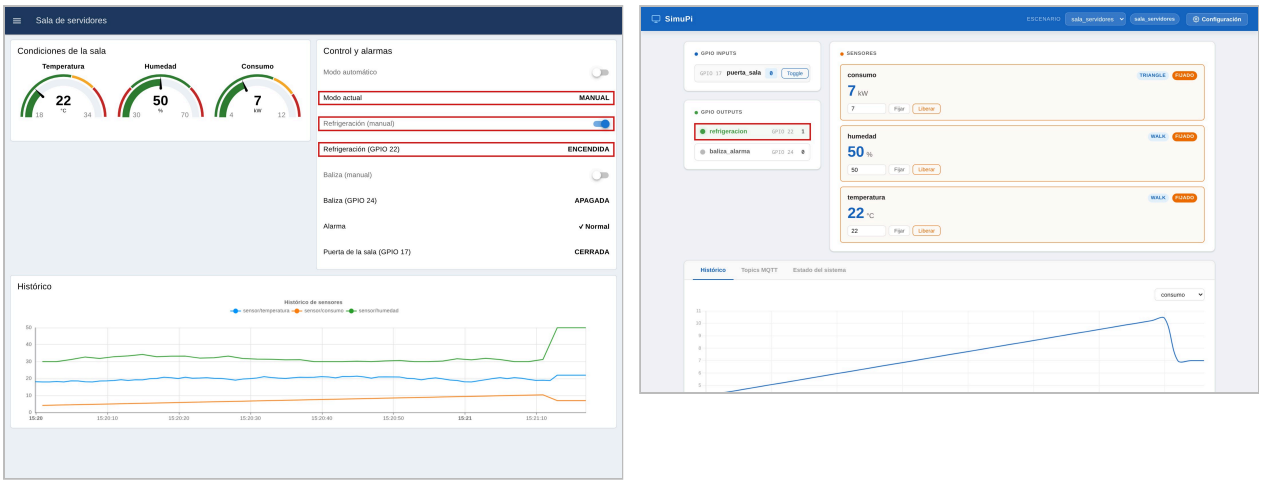


Figura 41: control manual de la refrigeración: dashboard y GPIO 22 en SimuPi.

b) Histéresis: encendido, mantenimiento y apagado

Con el modo AUTOMÁTICO activo fijamos la temperatura en 28 °C, después en 25,5 °C y por último en 23 °C:

```
curl -X POST localhost:5000/api/sensor/temperatura -H "Content-Type: application/json" -
d '{"value": 28}'
```

A 28 °C la refrigeración se enciende; a 25,5 °C, dentro de la banda, sigue encendida; a 23 °C se apaga.

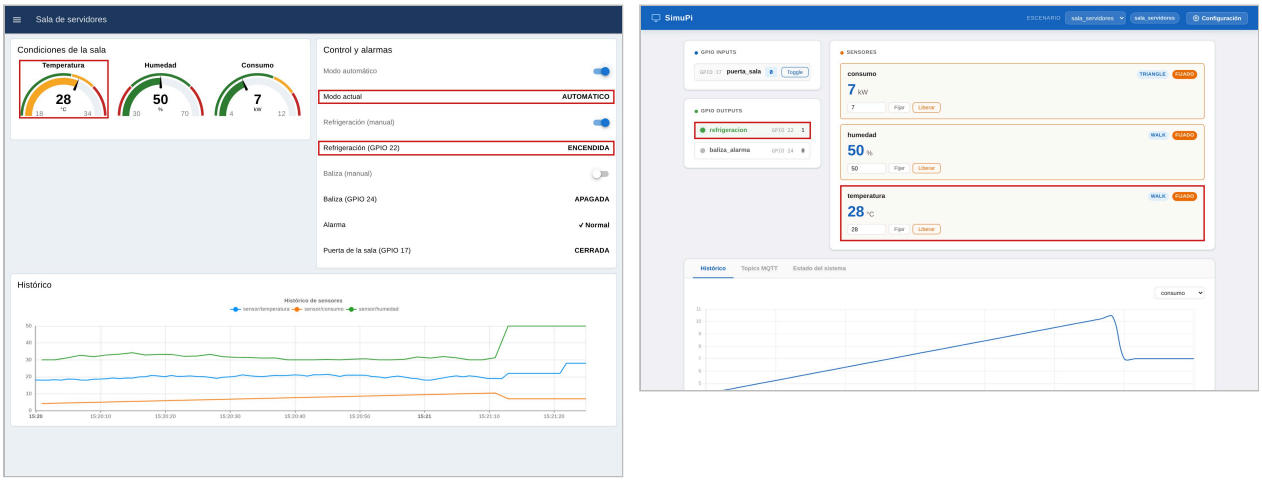


Figura 42: T = 28 °C en modo automático: la refrigeración se enciende.

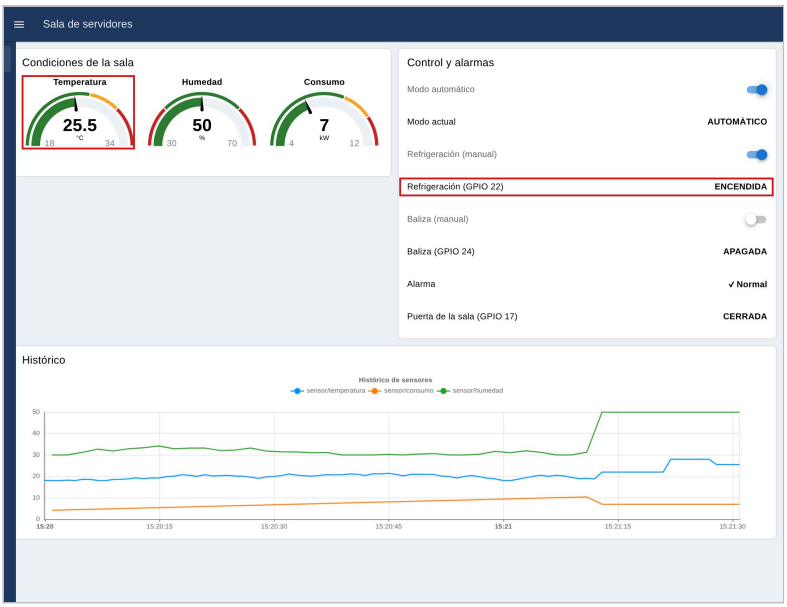


Figura 43: T = 25,5 °C: dentro de la banda de histéresis la refrigeración sigue encendida.

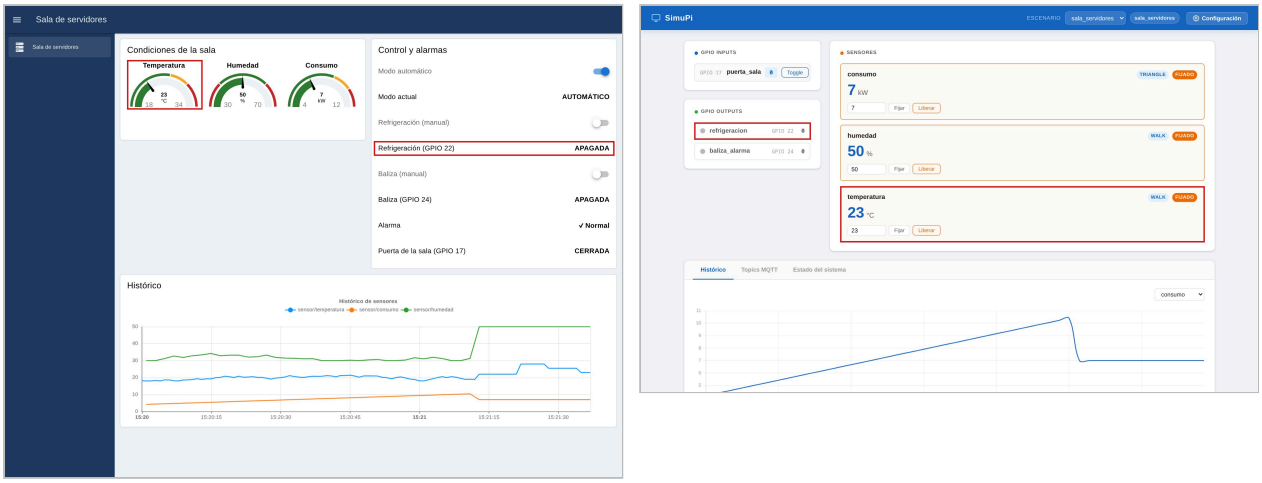


Figura 44: T = 23 °C: la refrigeración se apaga.

c) Alarma por temperatura

Con la temperatura en 31,5 °C el dashboard muestra “ALARMA: temperatura alta”, la baliza se activa y SimuPi pone en 1 los GPIO 22 y 24.

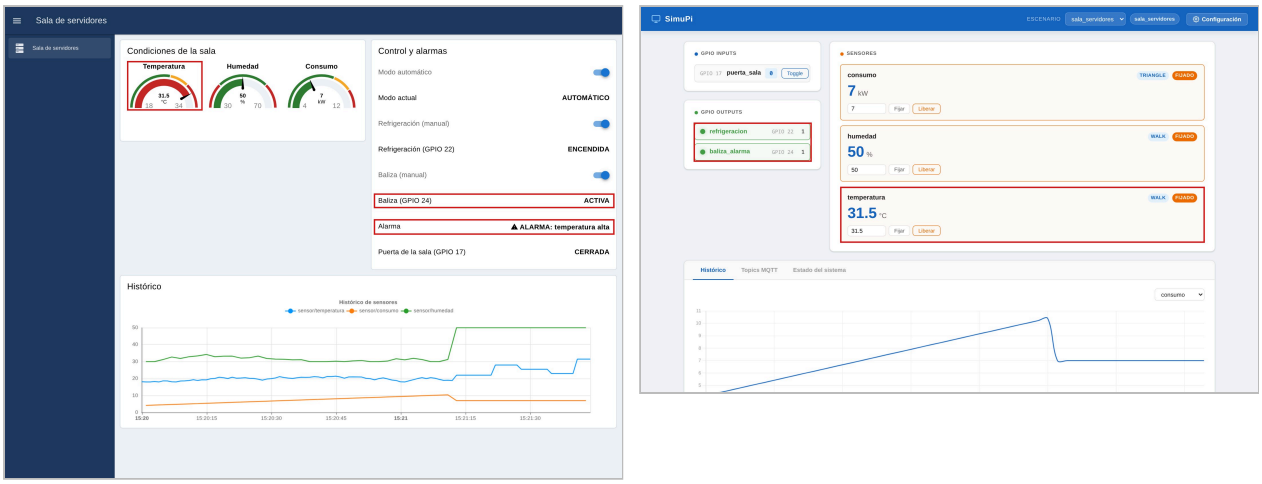


Figura 45: alarma por temperatura alta con la baliza activa.

d) Alarma por puerta abierta

Volvimos la temperatura a 22 °C y abrimos la puerta con `curl -X POST localhost:5000/api/gpio/17/toggle`. La alarma cambia de causa a “puerta abierta” y la baliza vuelve a activarse.

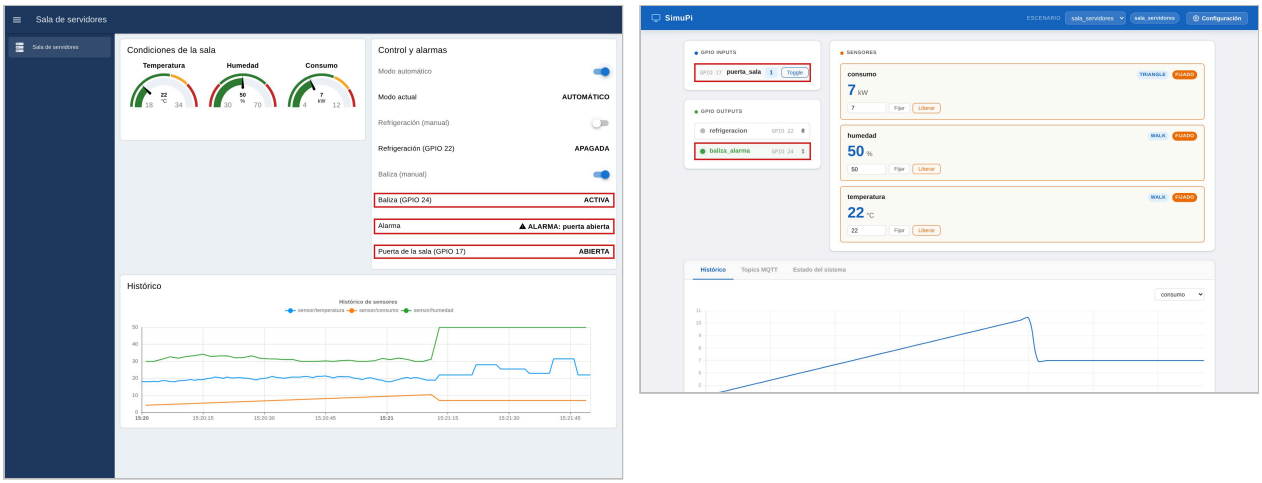


Figura 46: alarma por puerta abierta: GPIO 17 en 1 y baliza activa.

e) Vuelta a la normalidad

Al cerrar la puerta, con todos los valores dentro de rango, la alarma vuelve a “Normal” y la baliza se apaga.

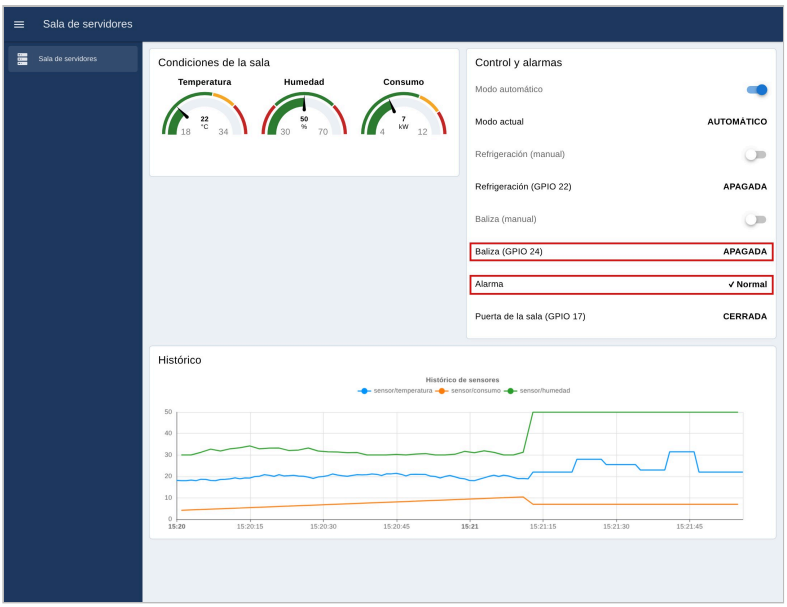


Figura 47: sin condiciones fuera de rango: alarma normal y baliza apagada.

f) Alarma en modo MANUAL

Repetimos la temperatura alta en modo MANUAL: aparece el texto de alarma, pero la baliza no se activa sola. Eso muestra que la automatización depende del modo, mientras que la alarma visible está siempre activa.

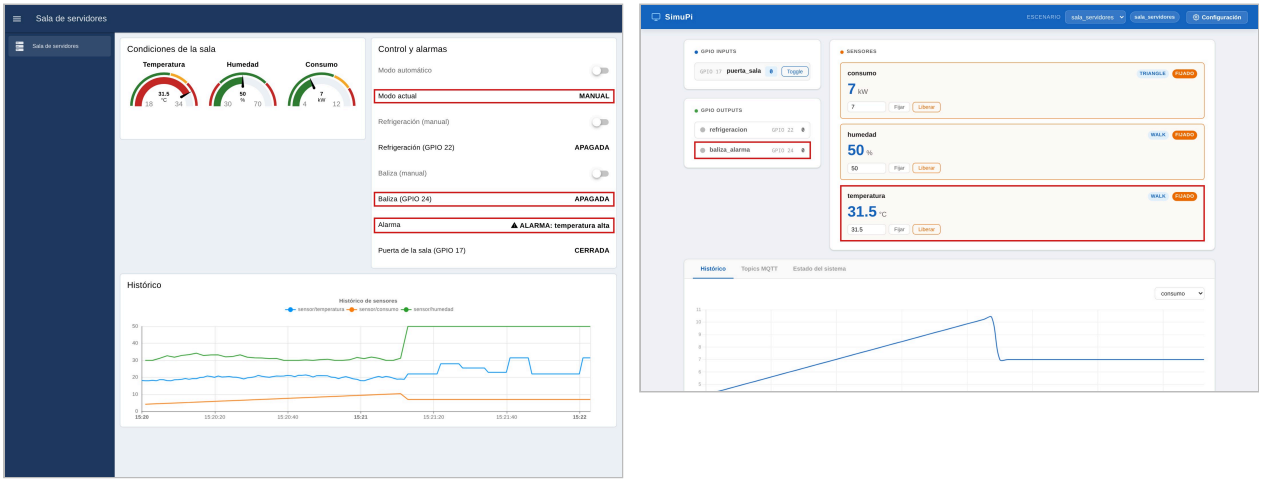


Figura 48: temperatura alta en modo manual: alarma visible y baliza apagada.

Conclusiones

- El bróker embebido de SimuPi cumple el mismo rol que AWS IoT en la Actividad 2: Node-RED no se comunica directamente con el simulador, sino que publica y se suscribe a tópicos en común.
- El límite de vCPU de la cuenta nos obligó a usar dos regiones. Eso impidió referenciar el grupo de seguridad de B desde A y nos obligó a usar la IP pública de A como bróker, lo que muestra por qué la consigna pide ambas instancias en la misma VPC y subred.
- Como SimuPi no publica el estado de las salidas, el dashboard refleja los comandos. Suscribirse a `gpio/out/#` y devolver el estado a los switches mantiene el dashboard coherente con lo que se

mandó.

- La histéresis evita que la refrigeración se encienda y apague en cada lectura cerca del umbral, y separar la alarma visible de la baliza permite que el modo MANUAL siga informando sin actuar.
- Reemplazar las instancias por contenedores Docker permitió reproducir la arquitectura de la actividad (dos servicios, bróker accesible solo para Node-RED) sin depender de AWS.